

AI VIET NAM

@aivietnam.edu.vn

Data Structure Review

List, Tuple, Set , Dictionary & Tree

Vinh Dinh Nguyen
PhD in Computer Science

Outline



➤ **Review: Tuples vs. Lists, 2D Lists**

➤ **IoU Calculation using Tuples**

➤ **NMS Using List and Tuples**

➤ **Review: Set and Dictionary**

➤ **One-Hot Encoding Using Set and Dictionary**

➤ **Review: Tree & K-DTree**

➤ **K-D Tree Implementation**

1D List Review

❖ A container that can contain elements

```
list_name = [element-1, ..., element-n]
```

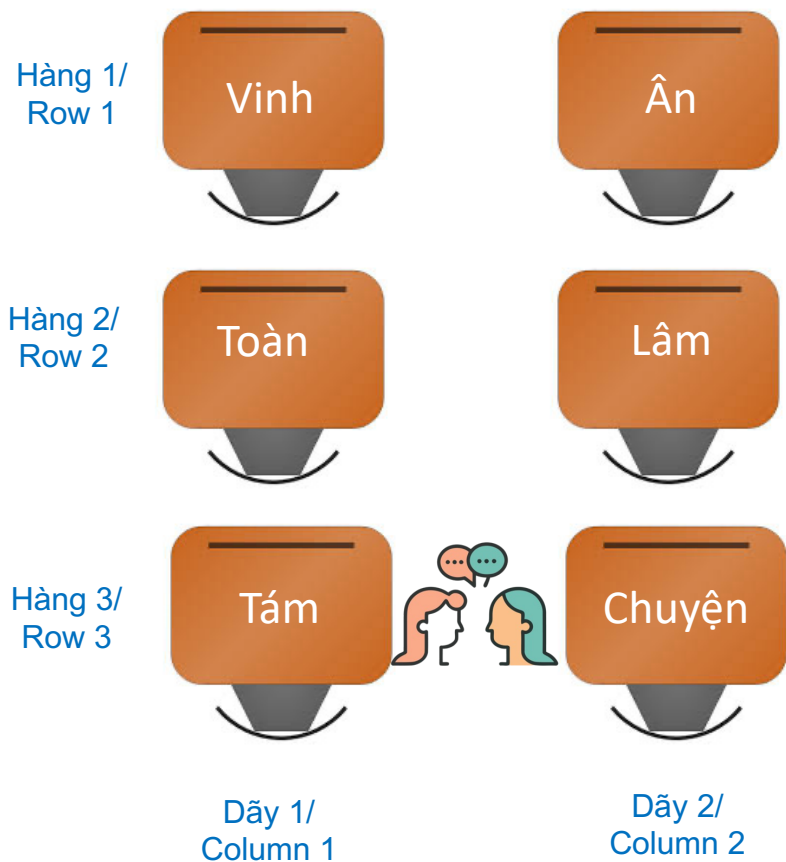
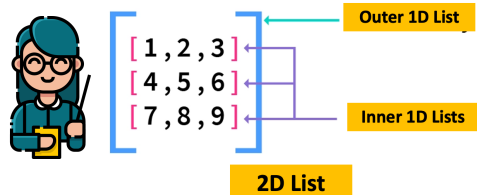
```
// create a list  
data = [6, 5, 7, 1, 9, 2]
```

data =	6	5	7	1	9	2
index	0	1	2	3	4	5

❑ Imputable

```
1. # danh sách trống  
2. emty_list = []  
3.  
4. # danh sách số tự nhiên nhỏ hơn 10  
5. my_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
6.  
7. # danh sách kết hợp nhiều kiểu dữ liệu  
8. mixedList = [True, 5, 'some string', 123.45]  
9. n_list = ["Happy", [2, 0, 1, 5]]  
10.  
11. #danh sách các loại hoa quả  
12. shoppingList = ['táo', 'chuối', 'cherries', 'dâu', 'mận']
```

From 1D to 2D List



Given that the teacher cannot remember students' names, please develop a program to help her find a student's name based on his/her seat location in the class.



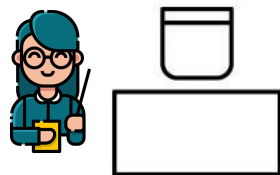
“Cô mời 2 bạn: **ngồi ở vị trí hàng thứ 3, dãy thứ 1**, và **ngồi ở vị trí hàng thứ 3, dãy thứ 2**” lên bảng giải thích về 2D List



“Cô mời bạn Tám và bạn Chuyện” lên bảng giải thích về 2D List



From 1D to 2D List



Hàng 1/
Row 1

Vinh

Ân

Hàng 2/
Row 2

Toàn

Lâm

Hàng 3/
Row 3

Tám

Chuyên

Dãy 1/
Column 1

Dãy 2/
Column 2

```
student_list = ["Vinh", "An", "Toan", "Lam", "Tam", "Chuyen"]
row_class = 3
col_class = 2
def find_student(student_list, r, c):
    index = (r-1)*col_class + (c-1)
    return student_list[index]
```

```
#Find student a location (3, 1)
print(find_student(student_list, 3, 1))
```

```
#Find student a location (3, 2)
print(find_student(student_list, 3, 2))
```

Tam
Chuyen



Are there any easier solutions? This one is really difficult for newcomers

From 1D to 2D List



Programmer View

	Column index 0	Column index 1
Row index 0	Vinh	An
Row index 1	Toan	Lam
Row index 2	Tam	Chuyen



Use 1D List

Vinh	An
------	----

Use 1D List

Toan	Lam
------	-----

Use 1D List

Tam	Chuyen
-----	--------

`student_list = [element1, element2, element3)`



Teacher View

	Column 1	Column 2
Row 1	Vinh	An
Row 2	Toan	Lam
Row 3	Tam	Chuyen

From 1D to 2D List



Teacher View

Column 1

Column 2

Row 1

Row 2

Row 3

Vinh	An
Toan	Lam
Tam	Chuyen

Column index 0

Column index 1

Row index 0

Row index 1

Row index 2

Vinh	An
Toan	Lam
Tam	Chuyen



Programmer View

```
row1_student = ["Vinh", "An"] #1D List
row2_student = ["Toan", "Lam"] #1D List
row3_student = ["Tam", "Chuyen"] #1D List
```

```
student_list = [row1_student, row2_student, row3_student] # 2D List
```

How to access element:
row 0 and column 1
row 0 and column 2

row1_retrieve = student_list[0]



Vinh

An

row1_retrieve[0]



Vinh

row1_retrieve[1]



An

student_list[0][0]



Vinh

student_list[0][1]



An

From 1D to 2D List

	Column 1	Column 2
Row 1	Vinh	An
Row 2	Toan	Lam
Row 3	Tam	Chuyen

Teacher View

	Column index 0	Column index 1
Row index 0	Vinh	An
Row index 1	Toan	Lam
Row index 2	Tam	Chuyen

Programmer View

```
row1_student = ["Vinh", "An"] #1D List
row2_student = ["Toan", "Lam"] #1D List
row3_student = ["Tam", "Chuyen"] #1D List

student_list = [row1_student, row2_student, row3_student] # 2D List
```

```
num_row = len(student_list)
num_col = len(student_list[0])
```

```
def find_student(student_list, r, c):
    return student_list[r-1][c-1]
```

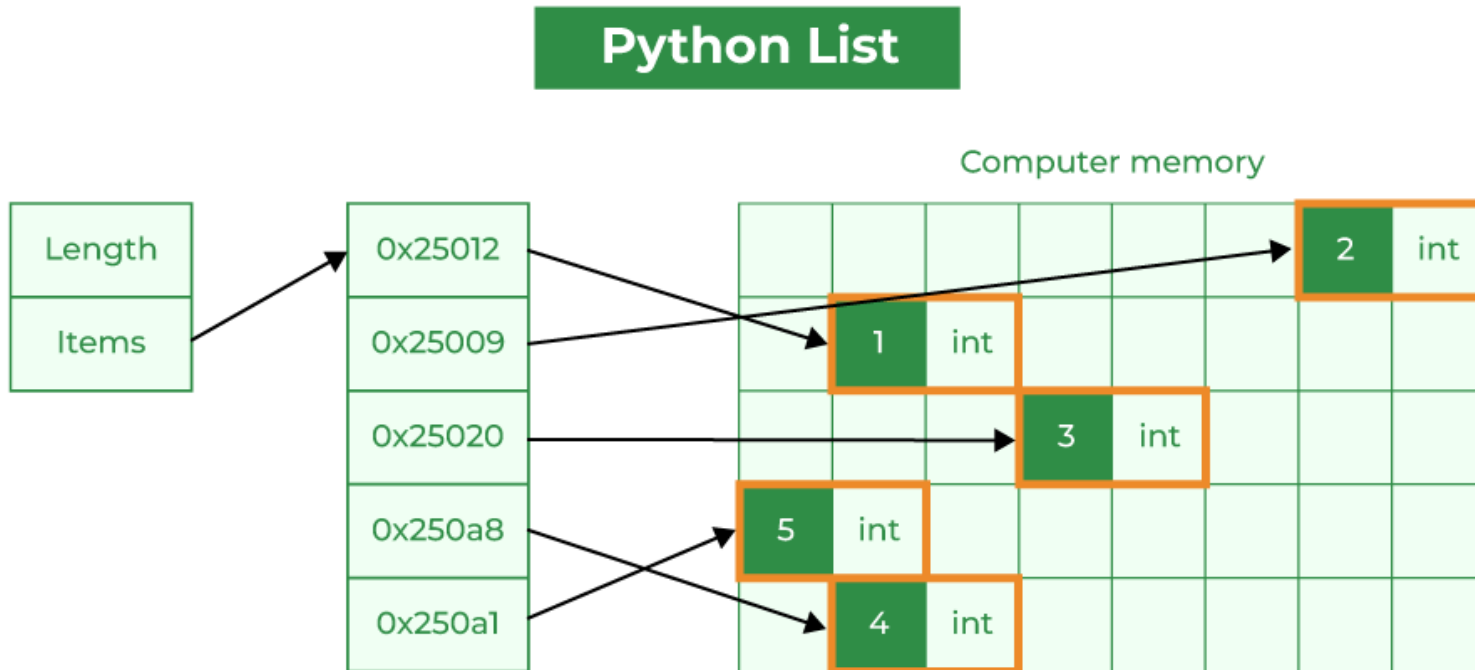
```
#Find student a location (3, 1)
print(find_student(student_list, 3, 1))
```

```
#Find student a location (3, 2)
print(find_student(student_list, 3, 2))
```

Tam
Chuyen

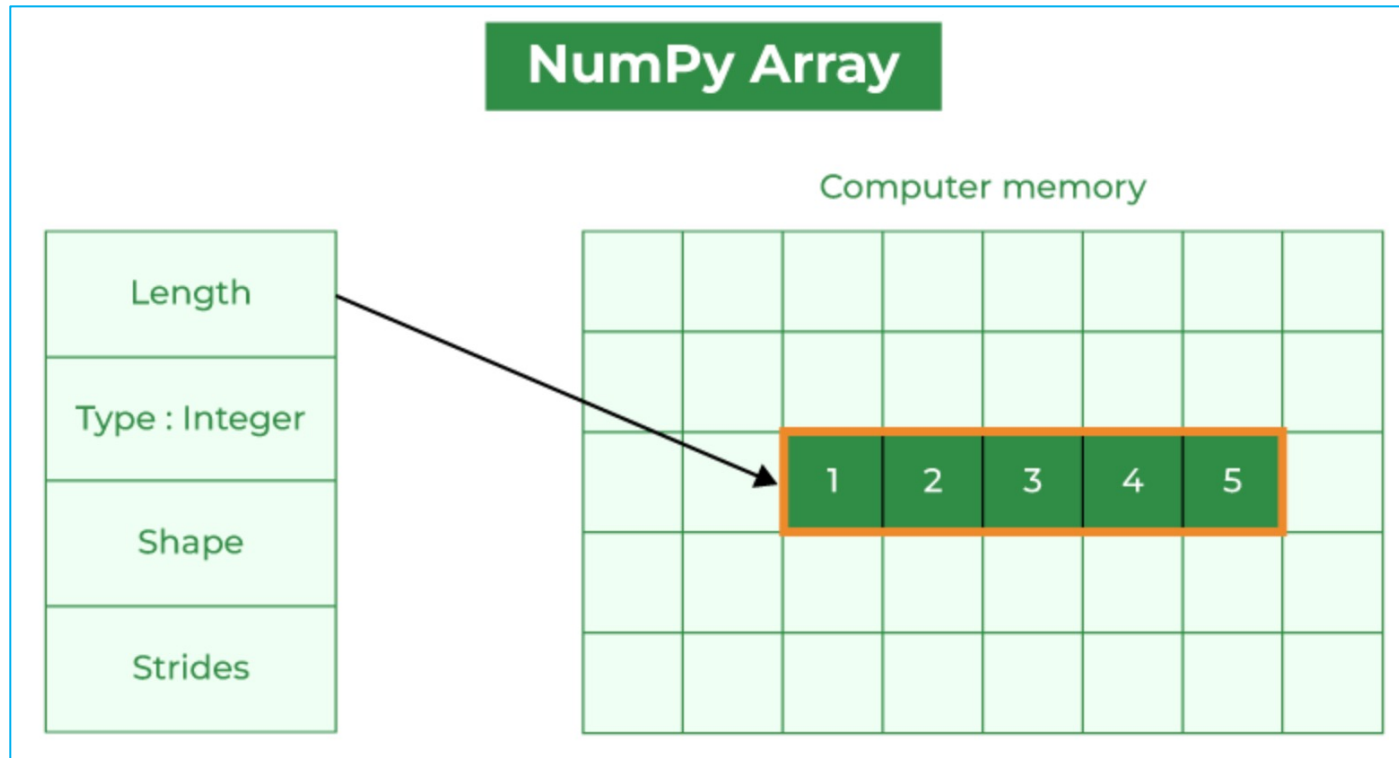
Thanks! This one is easy to understand

Python List Limitations



1. Element Overhead
2. Memory Fragmentation
3. Performance
4. Functionality

NumPy Array Motivation



- 1.Homogeneous Data
- 2.Fixed Data Type
- 3.Contiguous Memory
- 4.Performance

1D List/Array Multiplication

```
array_name = np.array[element-1, ..., element-n]
```

```
list_name = [element-1, ..., element-n]
```



```
def multiplizeList(size=1000):  
    # declaring lists  
    list1 = range(size)  
    list2 = range(size)  
  
    # capturing time before the multiplication of Python lists  
    initialTime = time.time()  
  
    # multiplying elements of both the lists and stored in another list  
    resultantList = [(a * b) for a, b in zip(list1, list2)]  
  
    # calculating execution time  
    print("Time taken by Lists to perform multiplication:",  
          (time.time() - initialTime),  
          "seconds")
```

```
def multiplizeArray(size=1000):  
    # declaring arrays  
    array1 = numpy.arange(size)  
    array2 = numpy.arange(size)  
  
    # capturing time before the multiplication of Numpy arrays  
    initialTime = time.time()  
  
    # multiplying elements of both the Numpy arrays and stored in another Numpy array  
    resultantArray = array1 * array2  
  
    # calculating execution time  
    print("Time taken by NumPy Arrays to perform multiplication:",  
          (time.time() - initialTime),  
          "seconds")
```

```
multiplizeList(100000)  
multiplizeArray(100000)
```

```
Time taken by Lists to perform multiplication: 0.025510549545288086 seconds  
Time taken by NumPy Arrays to perform multiplication: 0.0005364418029785156 seconds
```

2D List/Array Multiplication

❖ Python List Solution

```
def matrix_multiply(A, B):
    # Get the number of rows and columns for the input matrices
    rows_A, cols_A = len(A), len(A[0])
    rows_B, cols_B = len(B), len(B[0])

    # Ensure the number of columns in A is equal to the number of rows in B
    if cols_A != rows_B:
        raise ValueError("Number of columns in A must be equal to number of rows in B")

    # Initialize the result matrix with zeros
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

    # Perform the matrix multiplication
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]

    return result
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix} = \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$

$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$



Example matrices

```
A = [[1, 2, 3],
      [4, 5, 6]]
```

```
B = [[7, 8],
      [9, 10],
      [11, 12]]
```

Multiply the matrices

```
result = matrix_multiply(A, B)
```

Print the result

```
for row in result:
    print(row)
```



```
[58, 64]
[139, 154]
```

2D List/Array Multiplication

❖ NumPy Array Solution

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$

$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

```

import numpy as np

# Define the matrices
A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

B = np.array([
    [7, 8],
    [9, 10],
    [11, 12]
])

# Multiply the matrices using np.dot
result = np.dot(A, B)

# Alternatively, you can use the @ operator
# result = A @ B

# Print the result
print(result)

```

Tuple Review

❖ Structure

```
tuple_name = (element-1, ..., element-n)
```

() can be removed

```
1. t = 1, 2
2. print(t)
```

```
(1, 2)
```

Tuple with one element

```
1 var1 = (1 + 2) * 5
2 print(type(var1), ' ', var1)
3
4 var2 = (1)
5 print(type(var2), ' ', var2)
6
7 var3 = (1,)
8 print(type(var3), ' ', var3)
```

```
<class 'int'>    15
```

```
<class 'int'>    1
```

```
<class 'tuple'> (1,)
```

- ☐ Immutability
- ☐ Performance
- ☐ Multiple Return Values
- ☐ Unpacking

Outline



➤ **Review: Tuples vs. Lists, 2D Lists**

➤ **IoU Calculation using Tuples**

➤ **NMS Using List and Tuples**

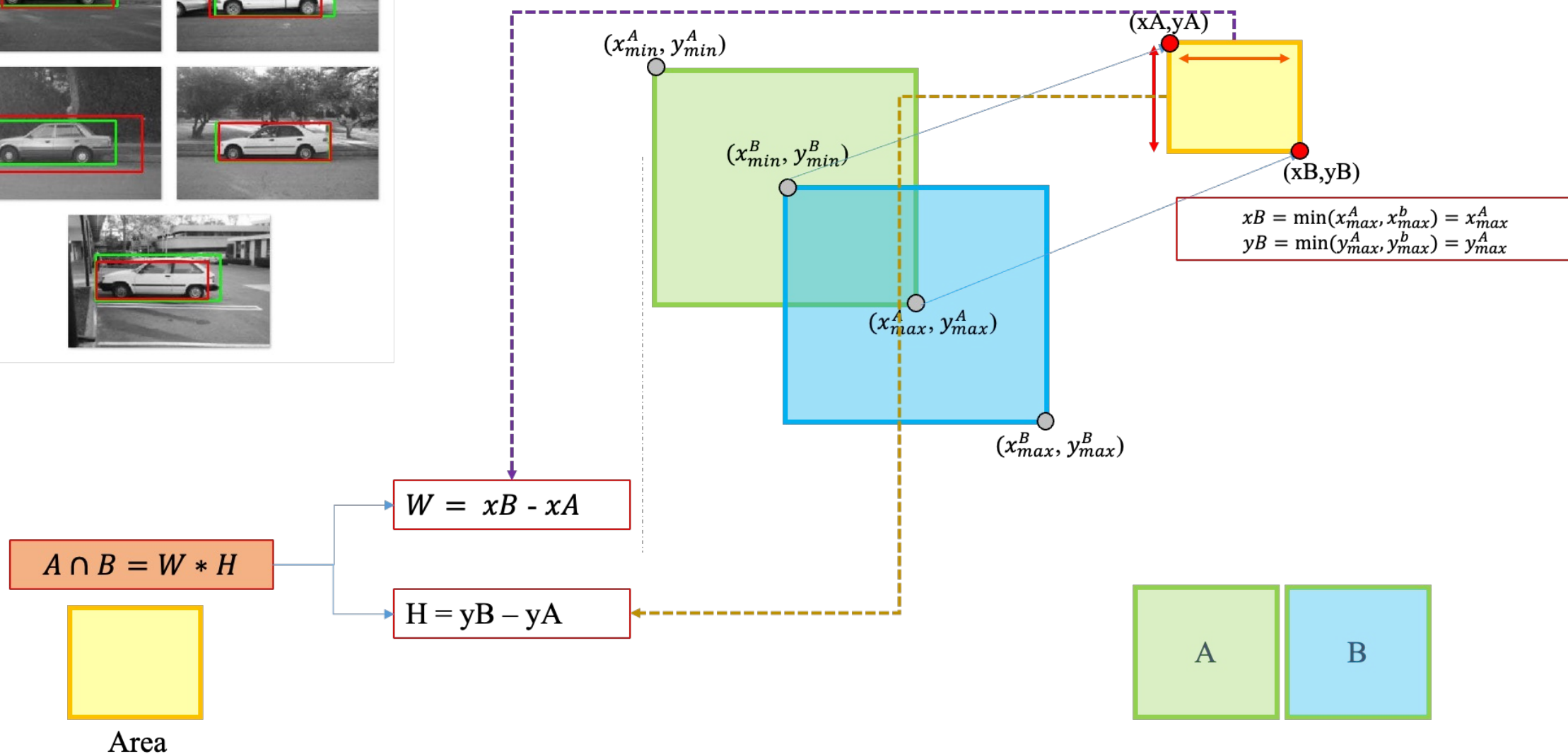
➤ **Review: Set and Dictionary**

➤ **One-Hot Encoding Using Set and Dictionary**

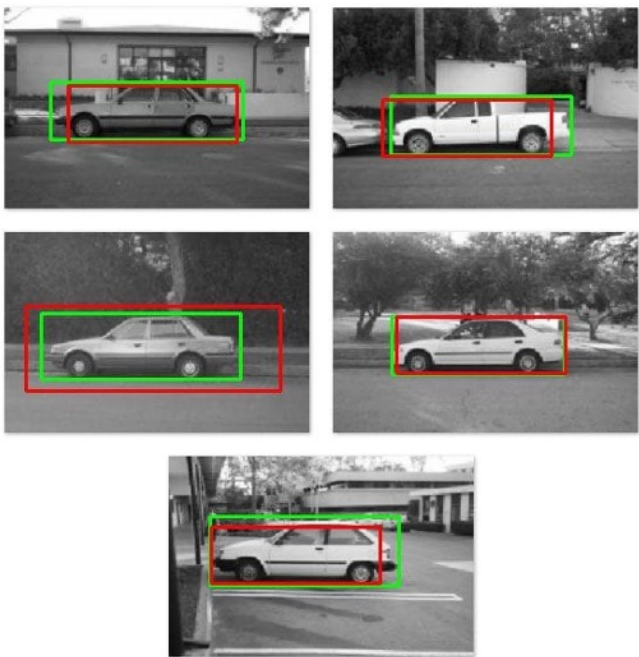
➤ **Review: Tree & K-DTree**

➤ **K-D Tree Implementation**

IOU Calculation Using Tuple



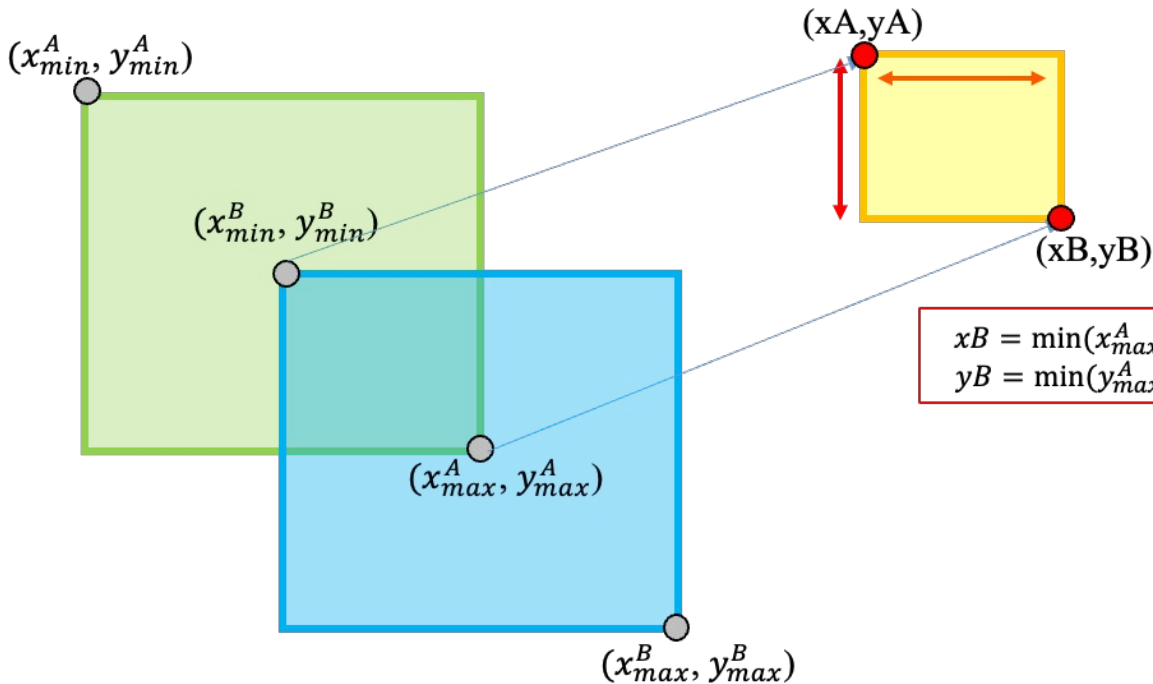
IOU Calculation Using Tuple



Giả sử ta có 2 boxes với thông tin như sau:

- Box A có tọa độ $(x_{min}^A, y_{min}^A, x_{max}^A, y_{max}^A) \Rightarrow \text{Tuple}$
- Box B có tọa độ $(x_{min}^B, y_{min}^B, x_{max}^B, y_{max}^B) \Rightarrow \text{Tuple}$

$$xA = \max(x_{min}^A, x_{min}^B) = x_{min}^B$$
$$yA = \max(y_{min}^A, y_{min}^B) = y_{min}^B$$



$$xB = \min(x_{max}^A, x_{max}^B) = x_{max}^A$$
$$yB = \min(y_{max}^A, y_{max}^B) = y_{max}^A$$

$(x_{max}^B - x_{min}^B) * (y_{max}^B - y_{min}^B)$

Area

Area

Area

$A \cup B =$

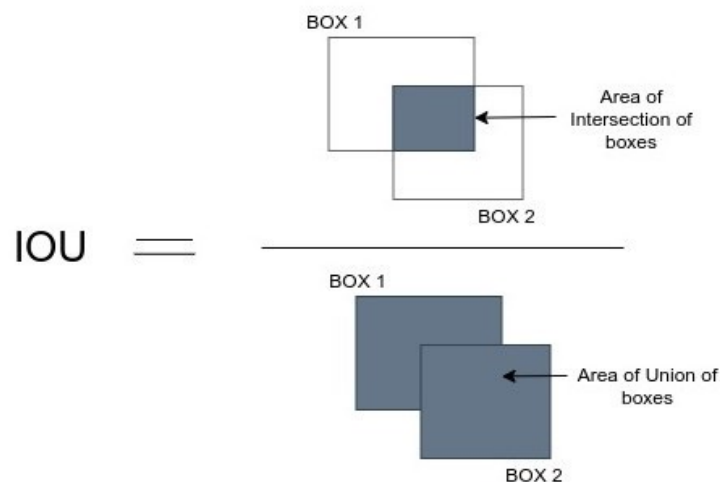
A

B

$(x_{max}^A - x_{min}^A) * (y_{max}^A - y_{min}^A)$



IOU Calculation Using Tuple



$$\frac{|A \cap B|}{|A \cup B|}$$

Tính diện tích của 2 box A, B

$$\text{area1} = (x_{\max}^A - x_{\min}^A) * (y_{\max}^A - y_{\min}^A)$$

$$\text{area2} = (x_{\max}^B - x_{\min}^B) * (y_{\max}^B - y_{\min}^B)$$

Tìm tọa độ của vùng giao nhau (Intersection)

$$xA = \max(x_{\min}^A, x_{\min}^B) = x_{\min}^B$$

$$yA = \max(y_{\min}^A, y_{\min}^B) = y_{\min}^B$$

$$xB = \min(x_{\max}^A, x_{\max}^B) = x_{\max}^A$$

$$yB = \min(y_{\max}^A, y_{\max}^B) = y_{\max}^A$$

Tính diện tích vùng giao nhau

$$W = xB - xA$$

$$H = yB - yA$$

$$\text{intersection_area} = w * h$$

Tính diện tích phần hợp nhau

$$\text{union_area} = \text{area1} + \text{area2} - \text{intersection_area}$$

Dựa trên phần giao và phần hợp để tính IoU

$$\text{IoU} = \text{intersection_area} / \text{union_area}$$

IOU Implementation

```

def computeIoU(boxA, boxB):
    # determine the (x, y)-coordinates of the intersection rectangle
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])

    # compute the area of intersection rectangle
    interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
    # compute the area of both the prediction and ground-truth
    # rectangles
    boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
    boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)
    # compute the intersection over union by taking the intersection
    # area and dividing it by the sum of prediction + ground-truth
    # areas - the interesection area
    iou = interArea / float(boxAArea + boxBArea - interArea)
    # return the intersection over union value
    return iou

```

Tính diện tích của 2 box A, B

$$\text{area1} = (x_{\max}^A - x_{\min}^A) * (y_{\max}^A - y_{\min}^A)$$

$$\text{area2} = (x_{\max}^B - x_{\min}^B) * (y_{\max}^B - y_{\min}^B)$$

Tìm tọa độ của vùng giao nhau (Intersection)

$$x_A = \max(x_{\min}^A, x_{\min}^B) = x_{\min}^B$$

$$y_A = \max(y_{\min}^A, y_{\min}^B) = y_{\min}^B$$

$$x_B = \min(x_{\max}^A, x_{\max}^B) = x_{\max}^A$$

$$y_B = \min(y_{\max}^A, y_{\max}^B) = y_{\max}^A$$

Tính diện tích vùng giao nhau

$$W = x_B - x_A$$

$$H = y_B - y_A$$

$$\text{intersection_area} = w * h$$

Tính diện tích phần hợp nhau

$$\text{union_area} = \text{area1} + \text{area2} - \text{intersection_area}$$

Dựa trên phần giao và phần hợp để tính IoU

$$\text{IoU} = \text{intersection_area} / \text{union_area}$$

Outline



➤ **Review: Tuples vs. Lists, 2D Lists**

➤ **IoU Calculation using Tuples**

➤ **NMS Using List and Tuples**

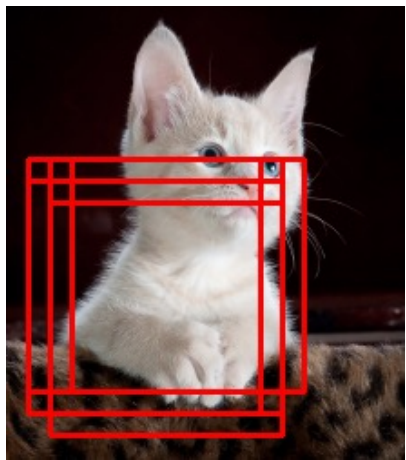
➤ **Review: Set and Dictionary**

➤ **One-Hot Encoding Using Set and Dictionary**

➤ **Review: Tree & K-DTree**

➤ **K-D Tree Implementation**

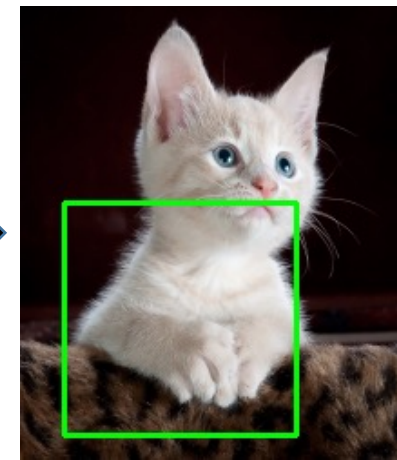
Non-Maxima Suppression: List & Tuple



List



Non Maximum Suppression



Tuple

Algorithm

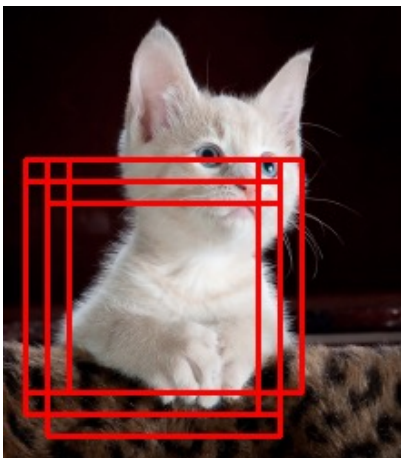
```
# import the necessary packages
import numpy as np
import cv2
from google.colab.patches import cv2_imshow

boundingBoxes = np.array([
    (12, 84, 140, 212, 0.3),
    (24, 84, 152, 212, 0.4),
    (36, 84, 164, 212, 0.5),
    (12, 96, 140, 224, 0.6),
    (24, 96, 152, 224, 0.7),
    (24, 108, 152, 236, 0.8)])
```

Step 1 : Select the prediction **S** with highest confidence score and remove it from **P** and add it to the final prediction list **keep**. (**keep** is empty initially).

Step 2 : Now compare this prediction **S** with all the predictions present in **P**. Calculate the IoU of this prediction **S** with every other predictions in **P**. If the IoU is greater than the threshold **thresh_iou** for any prediction **T** present in **P**, remove prediction **T** from **P**.

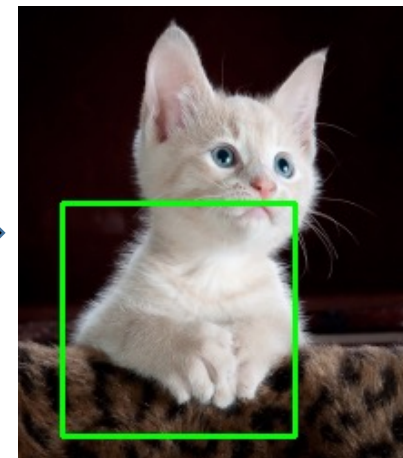
Step 3 : If there are still predictions left in **P**, then go to **Step 1** again, else return the list **keep** containing the filtered predictions.



```
# Example usage
boxes = [
    [12, 84, 140, 212],
    [24, 84, 152, 212],
    [36, 84, 164, 212],
    [12, 96, 140, 224],
    [24, 96, 152, 224],
    [24, 108, 152, 236]
]
```



Non Maximum Suppression



```
boxes = [
    [12, 84, 140, 212],
    [24, 84, 152, 212],
    [36, 84, 164, 212],
    [12, 96, 140, 224],
    [24, 96, 152, 224],
    [24, 108, 152, 236]
]

scores = [0.3, 0.4, 0.5, 0.6, 0.7]
iou_threshold = 0.3

kept_indices = non_max_suppression(boxes, scores, iou_threshold)
kept_boxes = [boxes[i] for i in kept_indices]

print("Kept indices:", kept_indices)
print("Kept boxes:", kept_boxes)
```

```
# Example usage
boxes = [
    [24, 108, 152, 236]
]
```

Q&A Time



Outline

➤ **Review: Tuples vs. Lists, 2D Lists**

➤ **IoU Calculation using Tuples**

➤ **NMS Using List and Tuples**

➤ **Review: Set and Dictionary**

➤ **One-Hot Encoding Using Set and Dictionary**

➤ **Review: Tree & K-DTree**

➤ **K-D Tree Implementation**



Set Review

❖ Create a set

Using curly brackets

```
1 # create a set
2 animals = {"cat", "dog", "tiger"}
3
4 print(type(animals))
5 print(animals)
```

```
<class 'set'>
{'dog', 'cat', 'tiger'}
```

Items with different data types

```
1 # create a set
2 a_set = {"cat", 5, True, 40.0}
3
4 print(type(a_set))
5 print(a_set)
```

```
<class 'set'>
{40.0, 'cat', 5, True}
```

Set comprehension

```
1 # set comprehension
2
3 a_set = {i*i for i in range(10)}
4 print(a_set)
```

```
{0, 1, 64, 4, 36, 9, 16, 49, 81, 25}
```



Dictionary Review

```
number_list = [11, 15, 35, 20, 18, 9]
```

Index

11	15	35	20	18	9
0	1	2	3	4	5

index

value

number_list[0] => 11



It's very hard to remember that the 1st element in a list has index 0, is there an easier way to remember this?



```
number_dict = {"One":11, "Two":15, "Three":35, "Four":20, "Five":18, "Six":9 }
```

key

11	15	35	20	18	9
"One"	Two	Three	Four	Five	Six

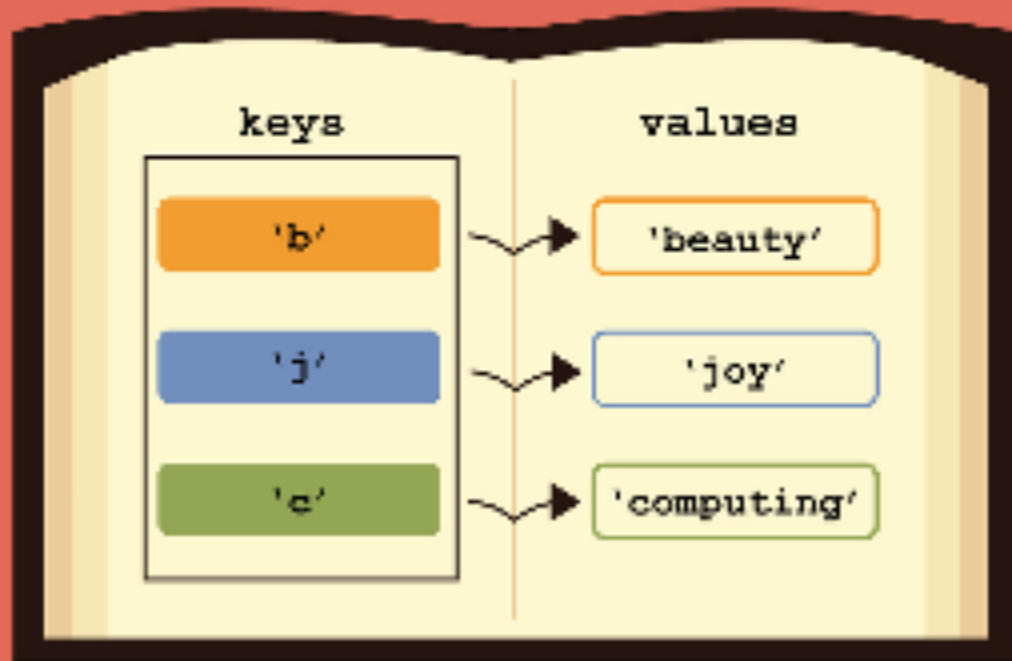
key

value

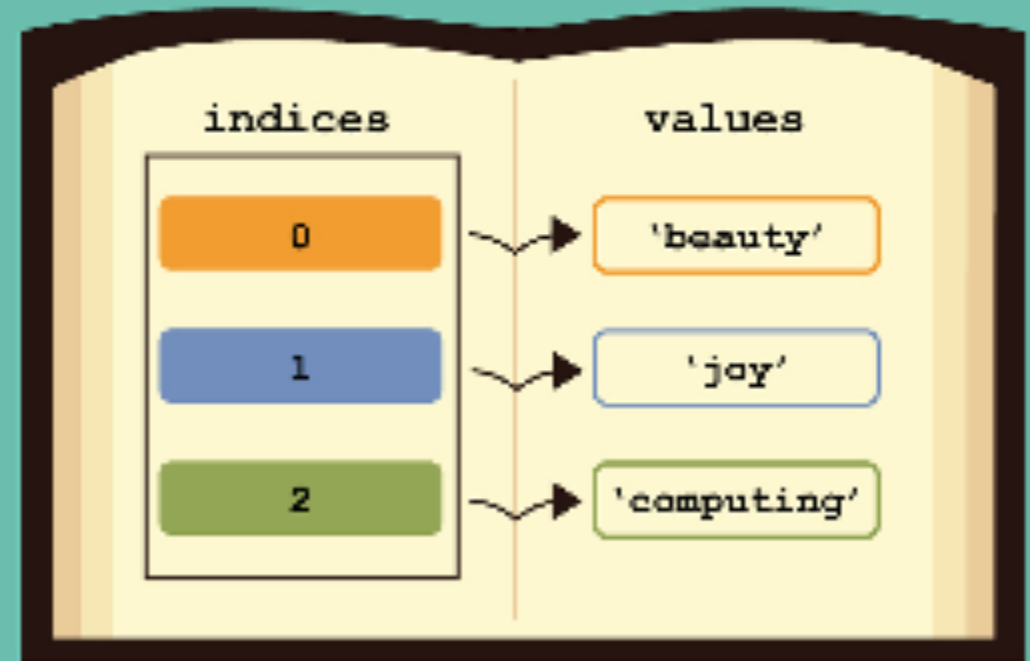
number_dict["One"] => 11

Dictionary Review

dictionaries

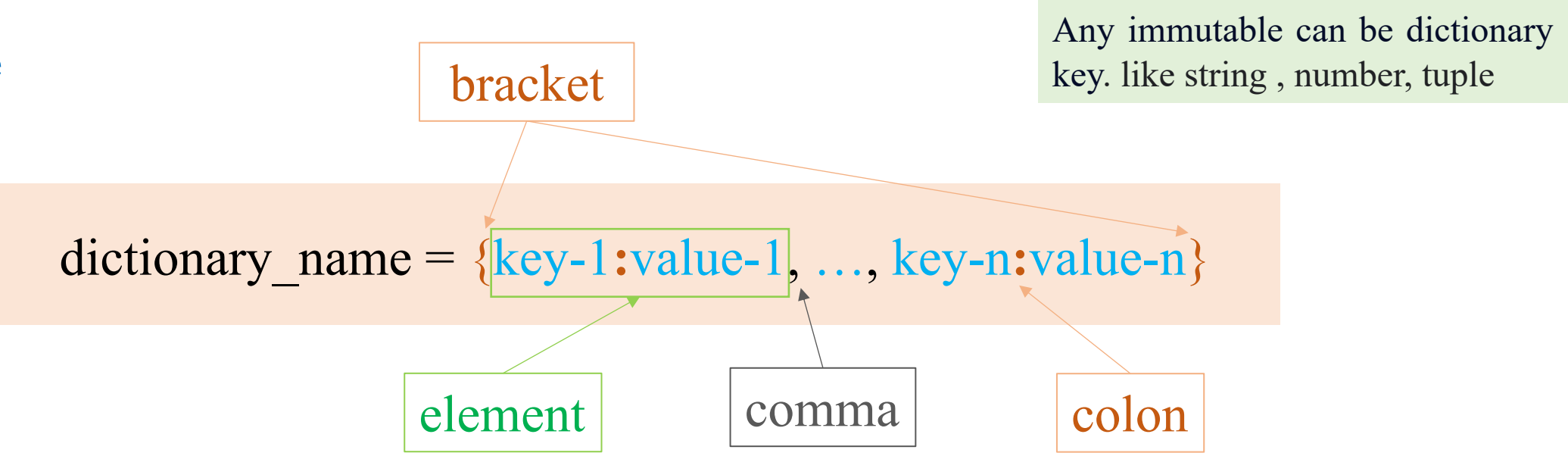


lists



Dictionary Review

❖ Structure



Create a dictionary

```
1 parameters = {'learning_rate': 0.1,
2               'optimizer': 'Adam',
3               'metric': 'Accuracy'}
4
5 print(parameters)
6 print(type(parameters))
```

```
{'learning_rate': 0.1, 'optimizer': 'Adam', 'metric': 'Accuracy'}
<class 'dict'>
```

Outline

➤ **Review: Tuples vs. Lists, 2D Lists**

➤ **IoU Calculation using Tuples**

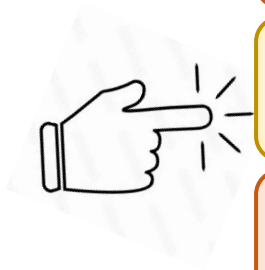
➤ **NMS Using List and Tuples**

➤ **Review: Set and Dictionary**

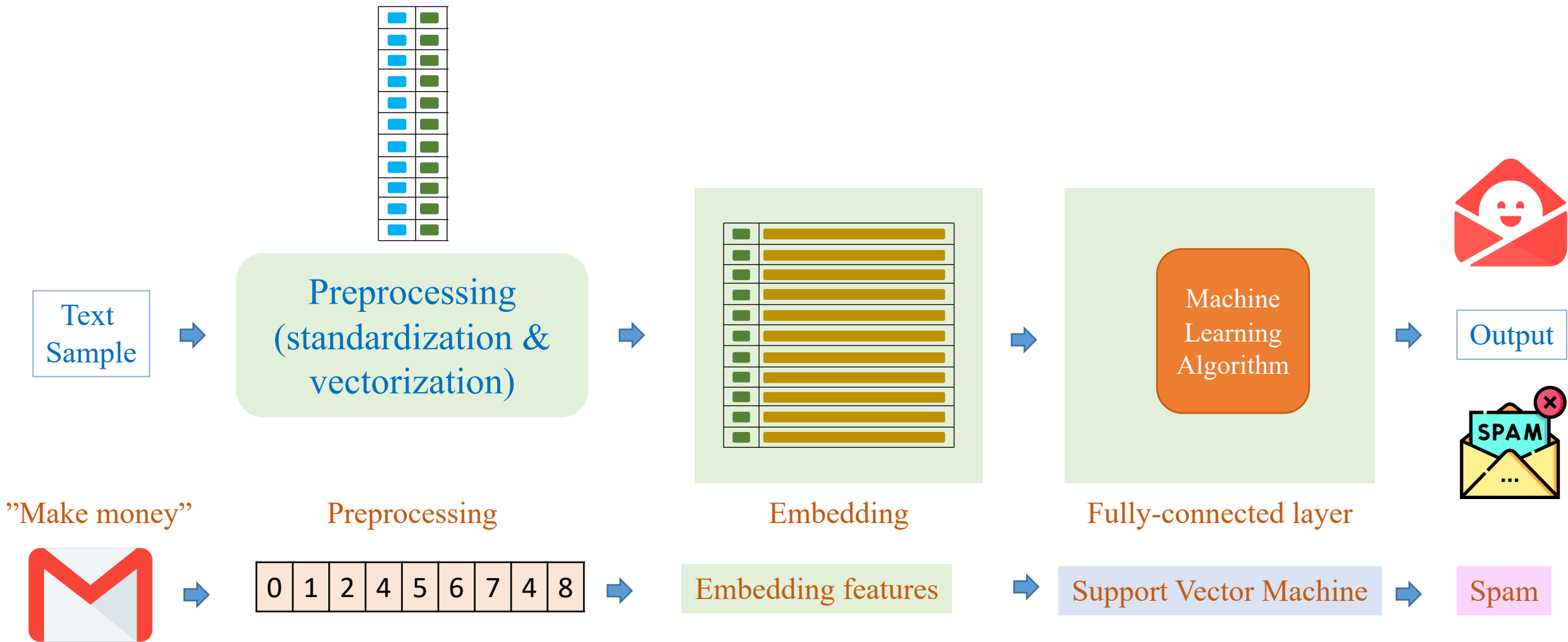
➤ **One-Hot Encoding Using Set and Dictionary**

➤ **Review: Tree & K-DTree**

➤ **K-D Tree Implementation**



❖ Text classification



How to Convert Categorical Data to Numerical Data?

- 1.Integer Encoding (already discussed in previous modules)
- 2.One-Hot Encoding

❖ Vocabulary Building

Python is a high-level, interpreted, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation. Python is dynamically-typed and garbage-collected. It supports multiple programming paradigms, including structured, object-oriented and functional programming.

Python Program
(using Set structure)

0 its

1 indentation

2 collected

3 supports

4 is

5 programming

6 garbage

7 purpose

8 it

9 including

10 and

11 typed

12 interpreted

13 dynamically

14 multiple

15 python

16 functional

17 readability

18 general

19 with

20 philosophy

21 object

22 level

23 paradigms

24 design

25 a

26 the

27 oriented

28 emphasizes

29 high

30 significant

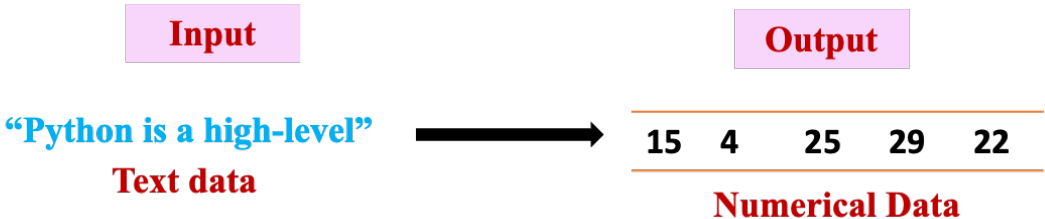
31 structured

32 language

33 code

34 use

35 of



Vocabulary

How to Convert Categorical Data to Numerical Data?

1.Integer Encoding (already discussed in previous modules)

2.One-Hot Encoding

Name	Integer Encoding
Ho Chi Minh	0
Da Nang	1
Ho Chi Minh	0
Da Nang	1
Can Tho	2

Integer Encoding

Can Tho	Da Nang	Ho Chi Minh
0	0	1
0	1	0
0	0	1
0	1	0
1	0	0

One-Hot Encoding

Set/Dictionary in ML Preprocessing

How to Convert Categorical Data to Numerical

1.Integer Encoding (already discussed in previous)

2.One-Hot Encoding

Can Tho	Da Nang	Ho Chi Minh
0	0	1
0	1	0
0	0	1
0	1	0
1	0	0

One-Hot Encoding

One-Hot Encoded Features:

```
[[0, 0, 1], [0, 1, 0], [0, 0, 1], [0, 1, 0], [1, 0, 0]]  
['Can Tho', 'Da Nang', 'Ho Chi Minh']
```

```
import numpy as np  
  
dataset = np.array(['Ho Chi Minh', 'Da Nang',  
                   'Ho Chi Minh', 'Da Nang', 'Can Tho'])  
  
# Initialize an empty set to store unique categories  
unique_categories = set()  
  
# Iterate through the dataset and add categories to the set  
for data in dataset:  
    unique_categories.add(data)  
  
# Convert the set to a sorted list to maintain order  
unique_categories = sorted(list(unique_categories))  
  
# Create a mapping from category to index  
category_to_index = {category: idx for idx, category in enumerate(unique_categories)}  
  
# Initialize an empty list to store the one-hot encoded features  
one_hot_encoded_features = []  
  
# Perform one-hot encoding  
for data in dataset:  
    one_hot_vector = [0] * len(unique_categories)  
    one_hot_vector[category_to_index[data]] = 1  
    one_hot_encoded_features.append(one_hot_vector)  
  
# Print the unique categories and one-hot encoded features  
print("One-Hot Encoded Features:\n", one_hot_encoded_features)  
print(unique_categories)
```

Q&A Time

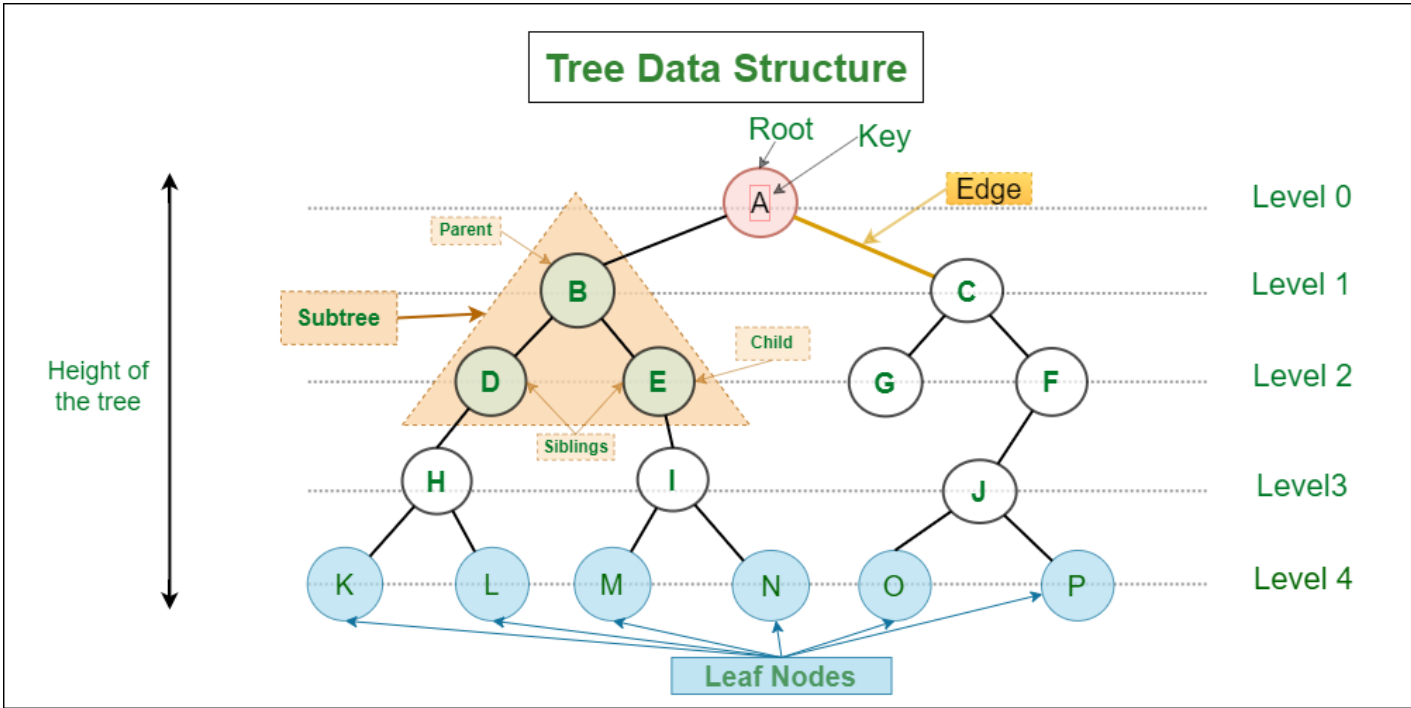


Outline

- **Review: Tuples vs. Lists, 2D Lists**
- **IoU Calculation using Tuples**
- **NMS Using List and Tuples**
- **Review: Set and Dictionary**
- **One-Hot Encoding Using Set and Dictionary**
- **Review: Tree & K-DTree**
- **K-D Tree Implementation**



A non-linear data structure where nodes are organized in a hierarchy



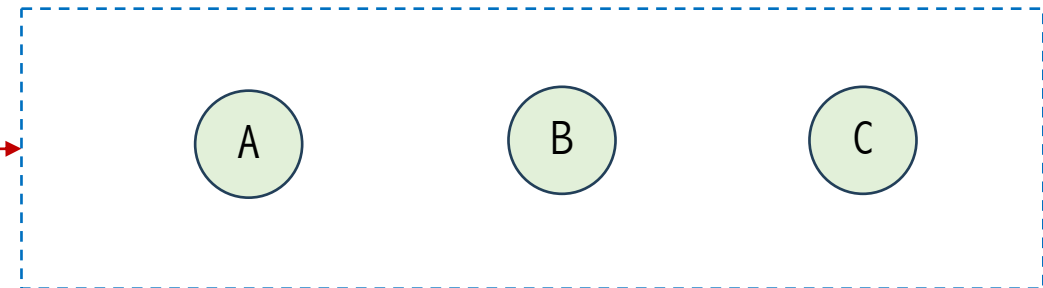
In the above figure, where do you think the root of the tree is?

Tree Review

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []
        self.parent = None
```

← Node of a Tree

```
a_node = Node("A")
b_node = Node("B")
c_node = Node("C")
```



Create nodes, A, B, and C

Tree Review

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []
        self.parent = None

    def add_child(self, child):
        child.parent = self
        self.children.append(child)

    def get_level(self): #to get level of each node
        level = 0
        p = self.parent
        while p:
            level += 1
            p = p.parent

        return level

    def print_tree(self):
        space = ' ' * self.get_level() * 3
        prefix = space + '|__' if self.parent else ''
        print(prefix + self.data)#add prefix
        if self.children:
            for child in self.children:
                child.print_tree()
```

```
def create_tree():
    a_node = TreeNode("A")
    b_node = TreeNode("B")
    c_node = TreeNode("C")
    d_node = TreeNode("D")
    e_node = TreeNode("E")
    f_node = TreeNode("F")
    g_node = TreeNode("G")

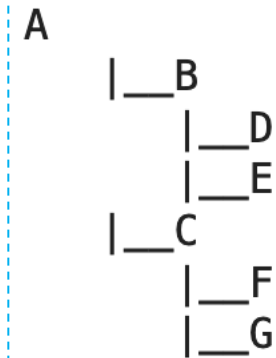
    a_node.add_child(b_node)
    a_node.add_child(c_node)

    b_node.add_child(d_node)
    b_node.add_child(e_node)

    c_node.add_child(f_node)
    c_node.add_child(g_node)

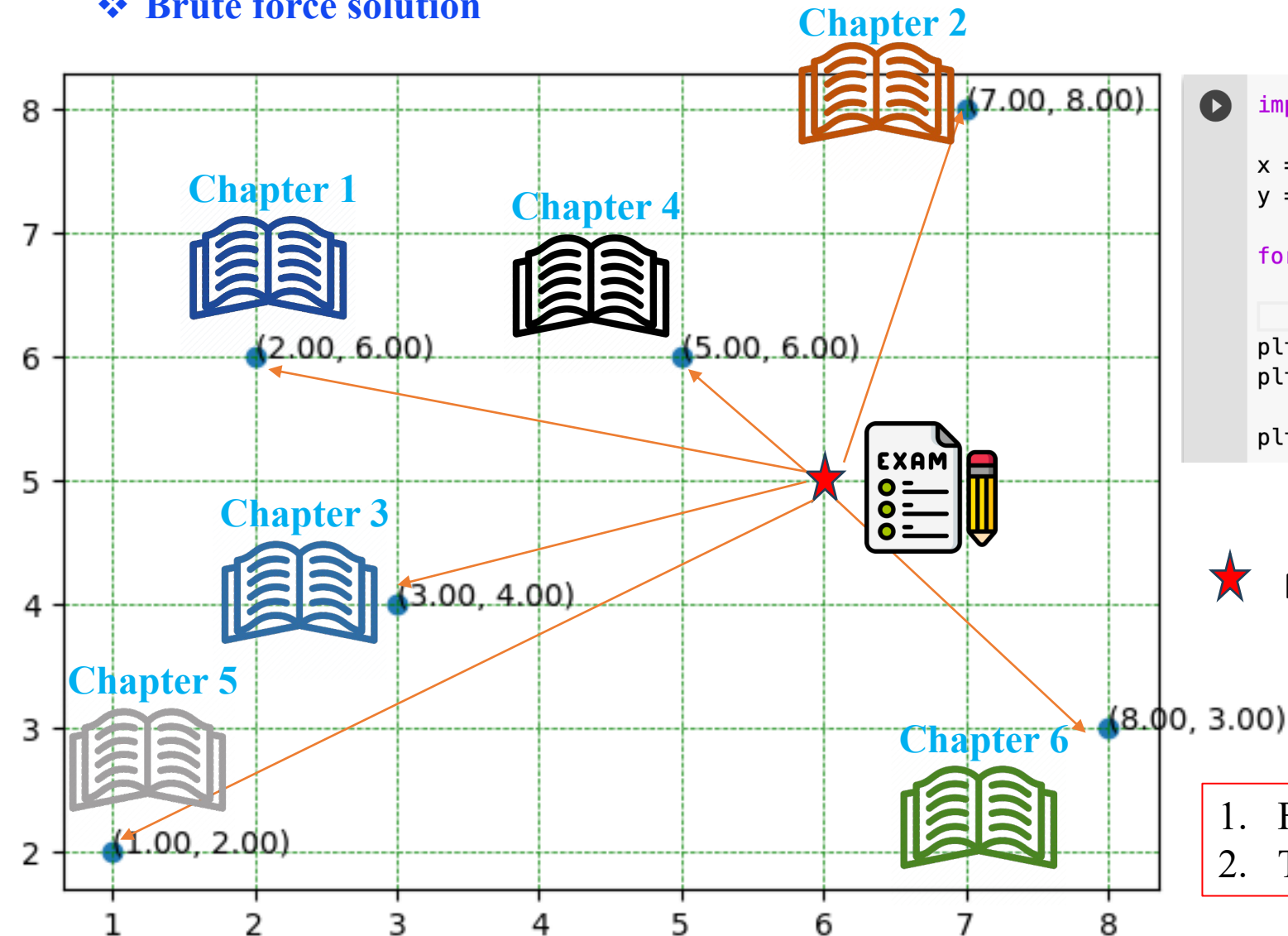
    return a_node
```

```
tree = create_tree()
tree.print_tree()
```



Classification Problem

❖ Brute force solution



```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 5, 7, 8]
```

```
y = [2, 6, 4, 6, 8, 3]
```

```
for xy in zip(x, y):
```

```
    plt.annotate('({:.2f}, {:.2f})'.format(xy[0], xy[1]), xy=xy)
```

```
plt.scatter(x, y)
```

```
plt.grid(color = 'green', linestyle = '--', linewidth = 0.5)
```

```
plt.show()
```



New datapoint

1. Find distance of each point in the dataset
2. The point with lowest distance would be nears

K-D Tree Implementation Solution

Algorithm 1: K-D Tree Construction

Function *kdtree*(*pointList*, *depth*):

Input: a list of points, *pointList*, and an integer, *depth* ;

Output: a k-d tree rooted at the median point of *pointList* ;

/ Select axis based on depth so that axis cycles
through all valid values */*

let axis := depth mod k;

/ Sort point list and choose median as pivot element
/

select median by axis from pointList;

/ Create node and construct subtree */*

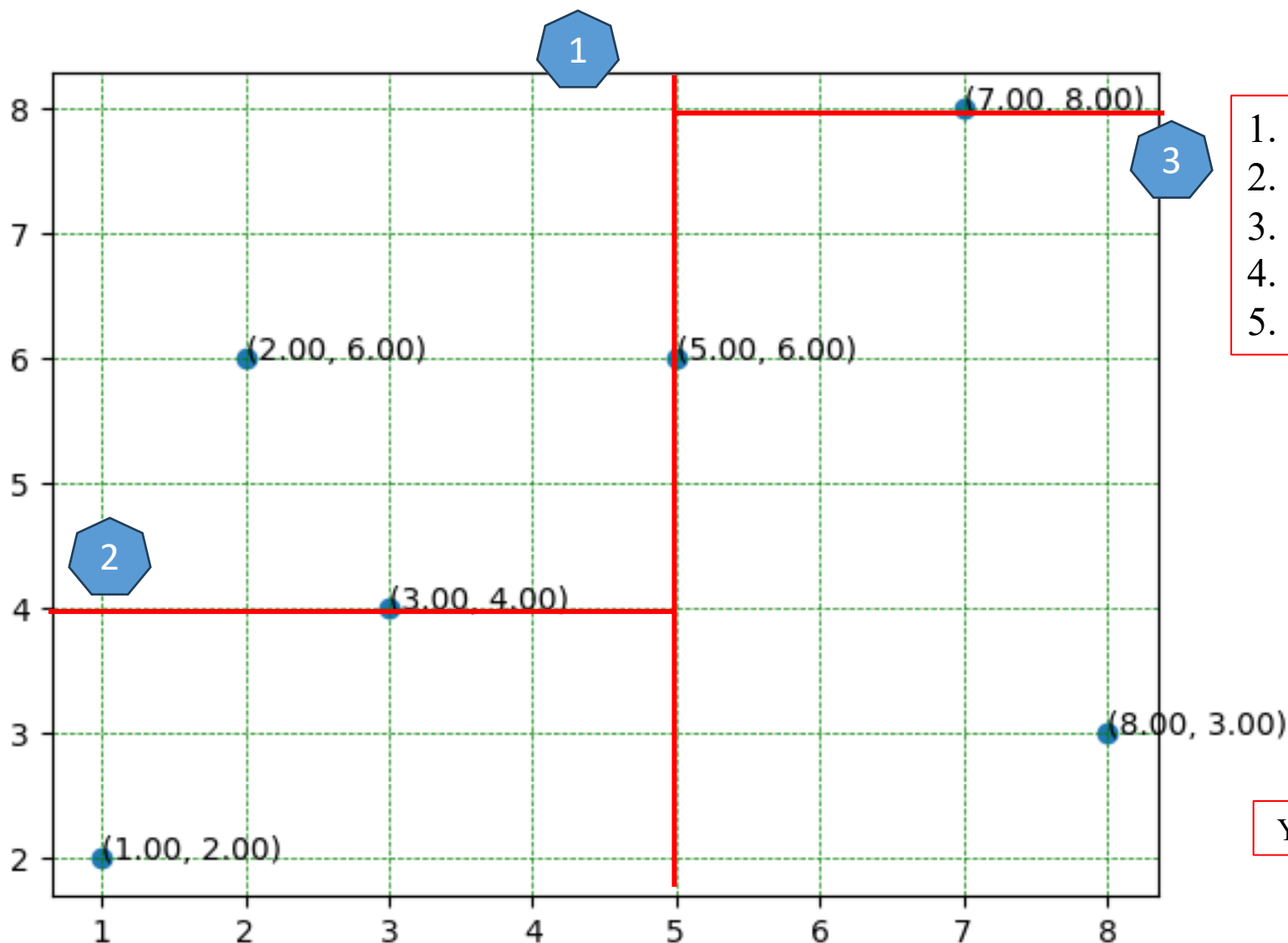
let node.location := median;

*let node.leftChild := kdtree(points in pointList before median,
depth + 1);*

*let node.rightChild := kdtree(points in pointList after median,
depth + 1);*

return *node*;

K-D Tree Implementation Solution



1. Pick any one feature at random
2. Find median
3. Split dataset in approximate equal halves
4. Pick next feature and repeat step #2,3
5. Continue until all data points are partitioned

X = 1, 2, 3, 5, 7, 8; median = 5



Left

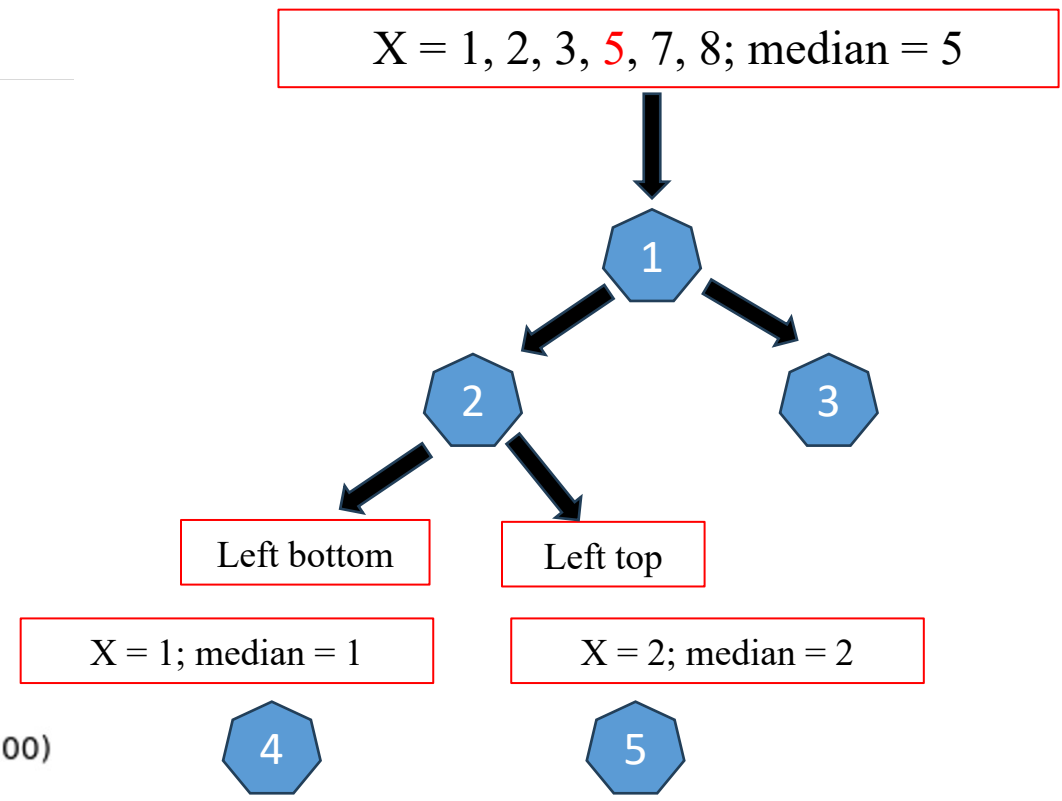
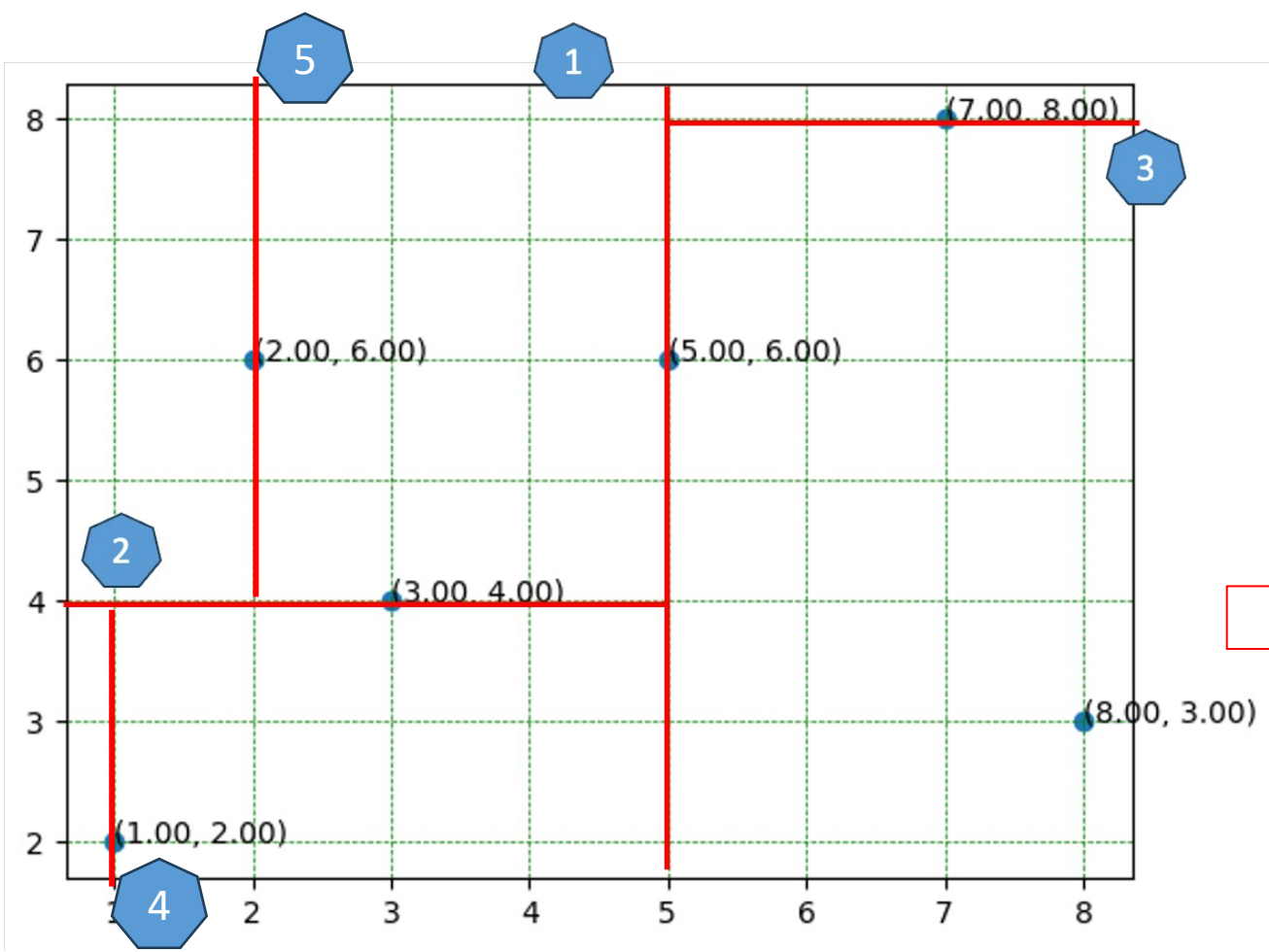
Right

Y = 2, 4, 6; median = 4

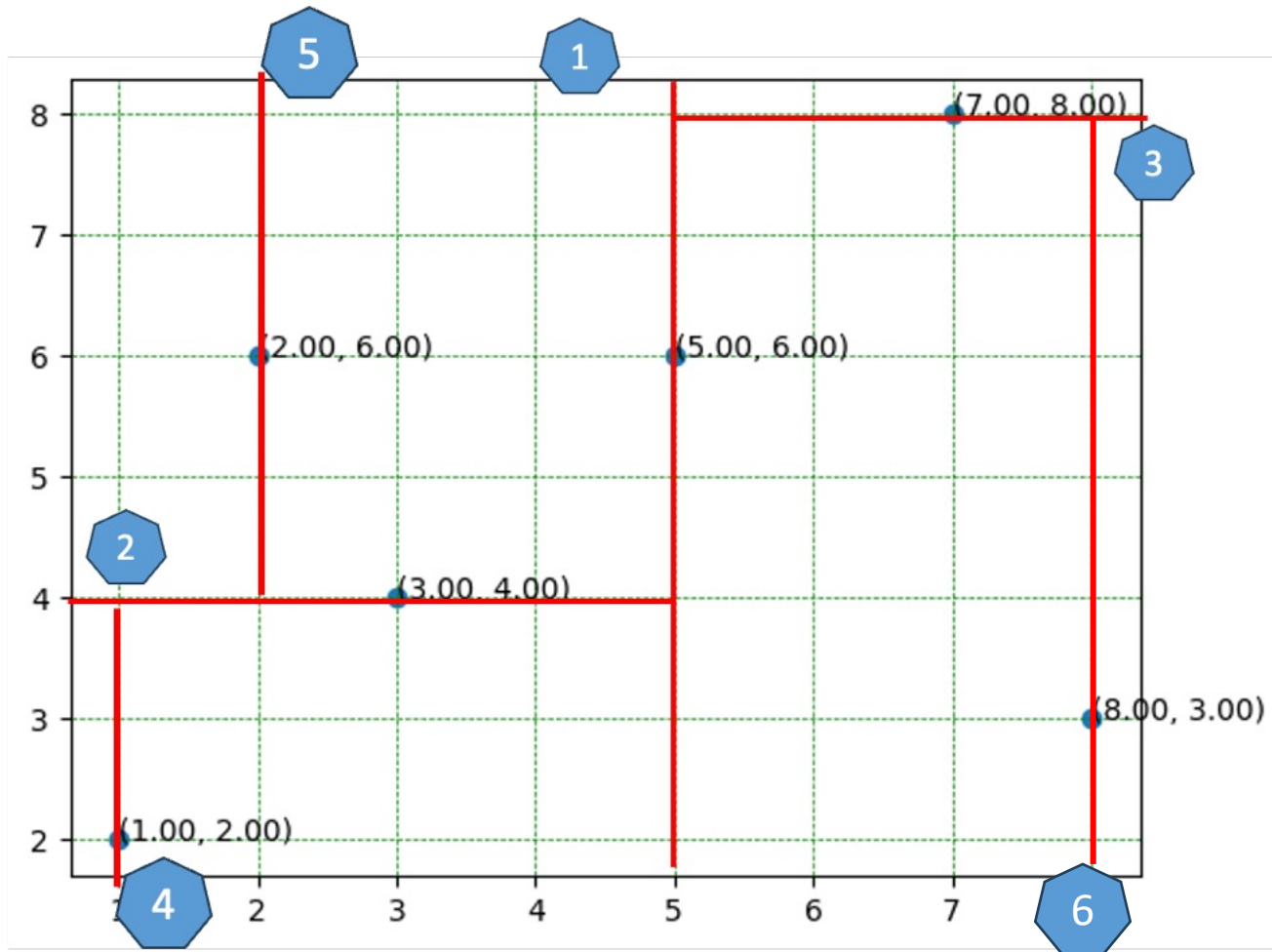
Y = 3, 8; median = 8



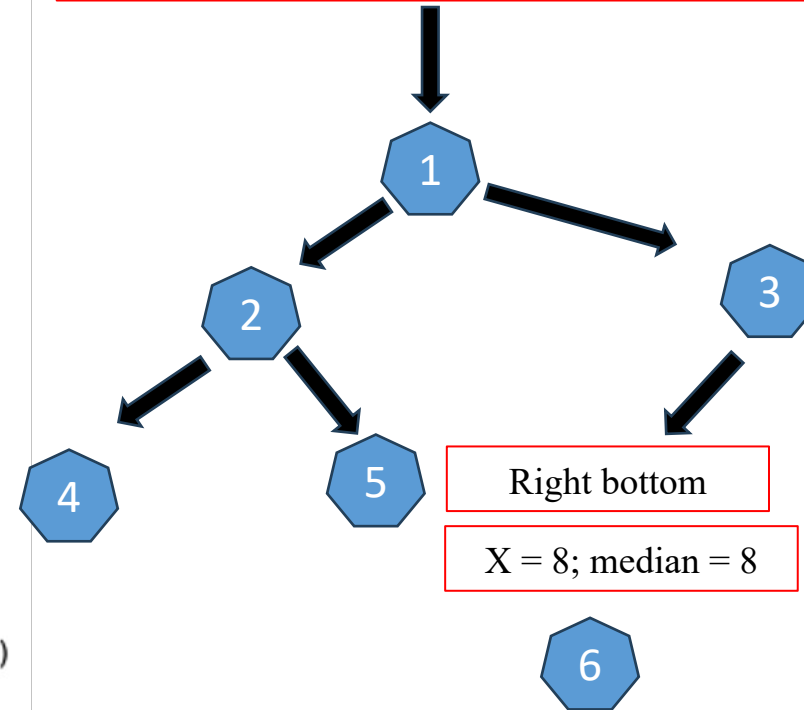
K-D Tree Implementation Solution



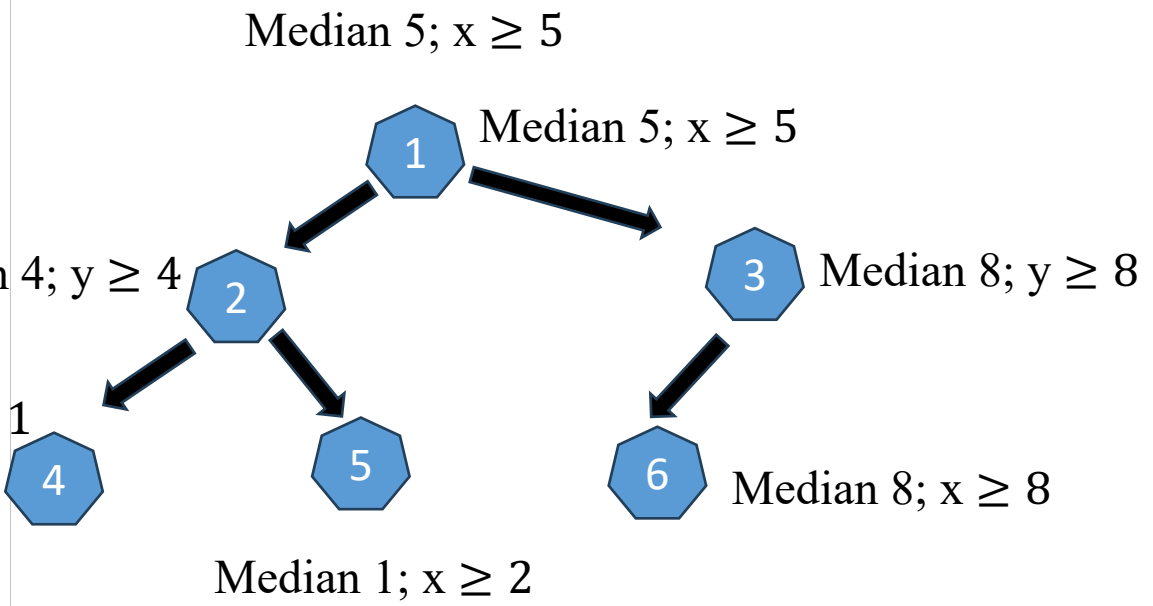
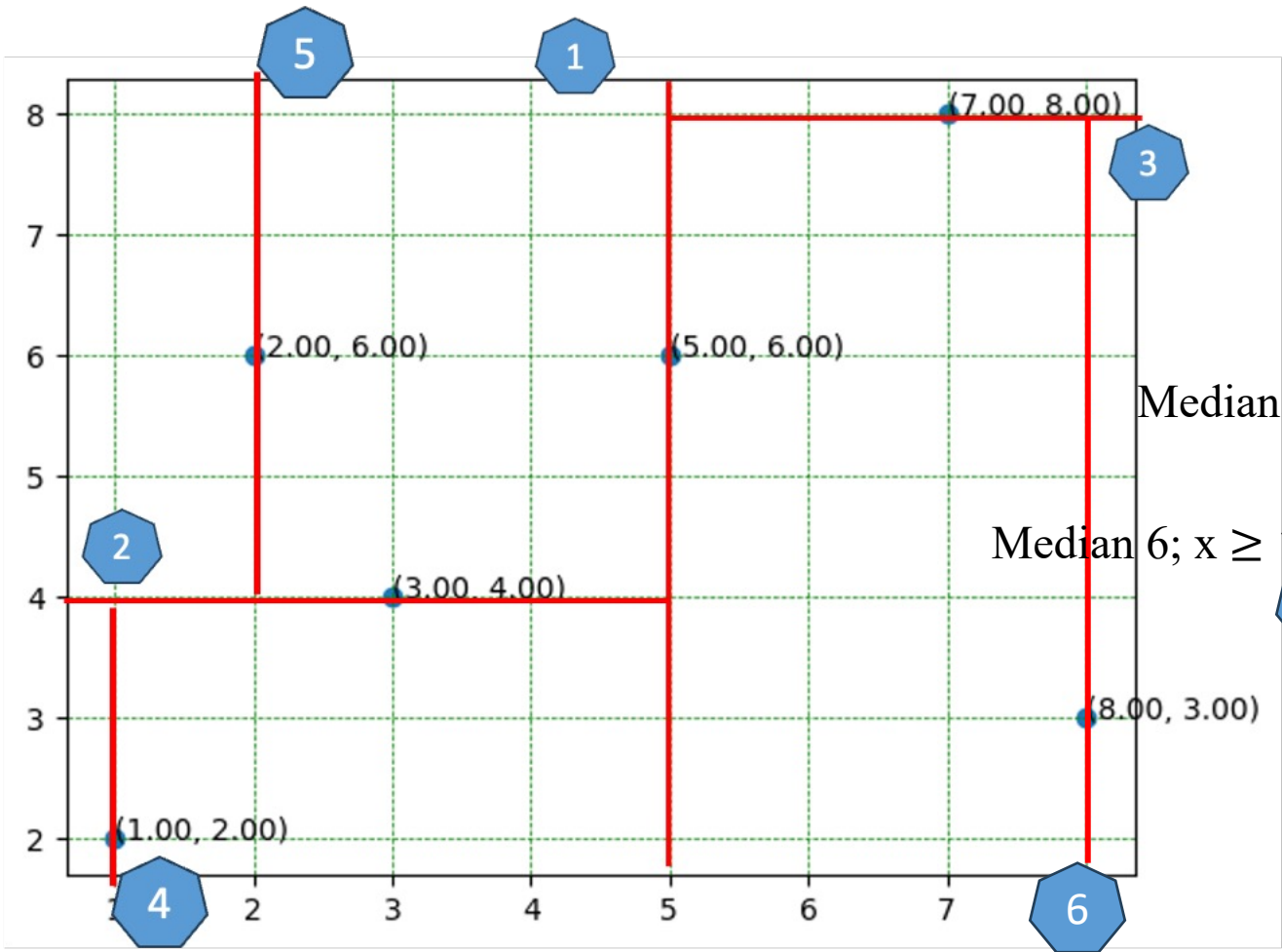
K-D Tree Implementation Solution



$X = 1, 2, 3, 5, 7, 8$; median = 5

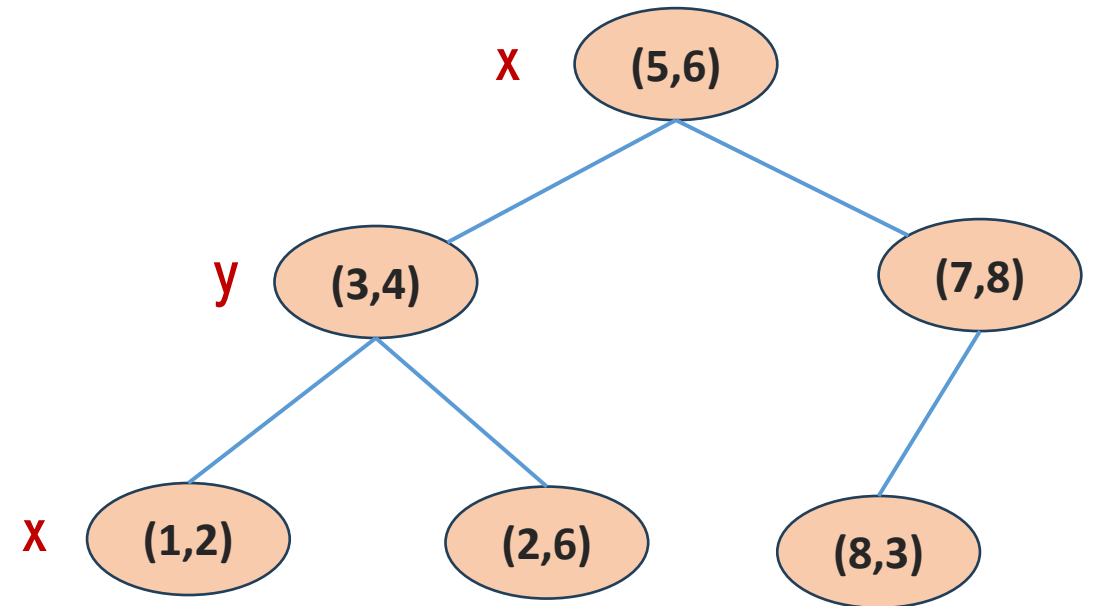
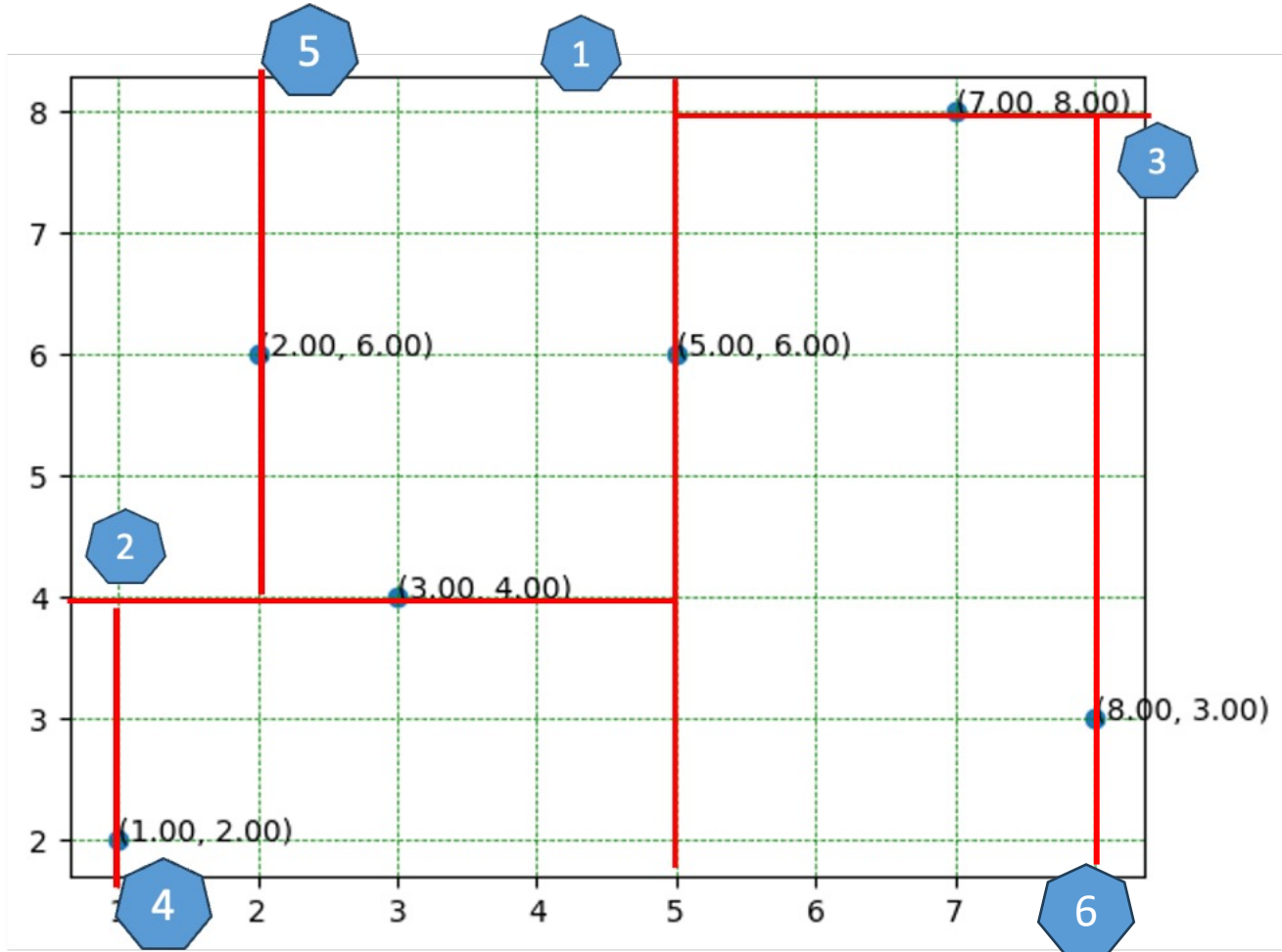


K-D Tree Implementation Solution

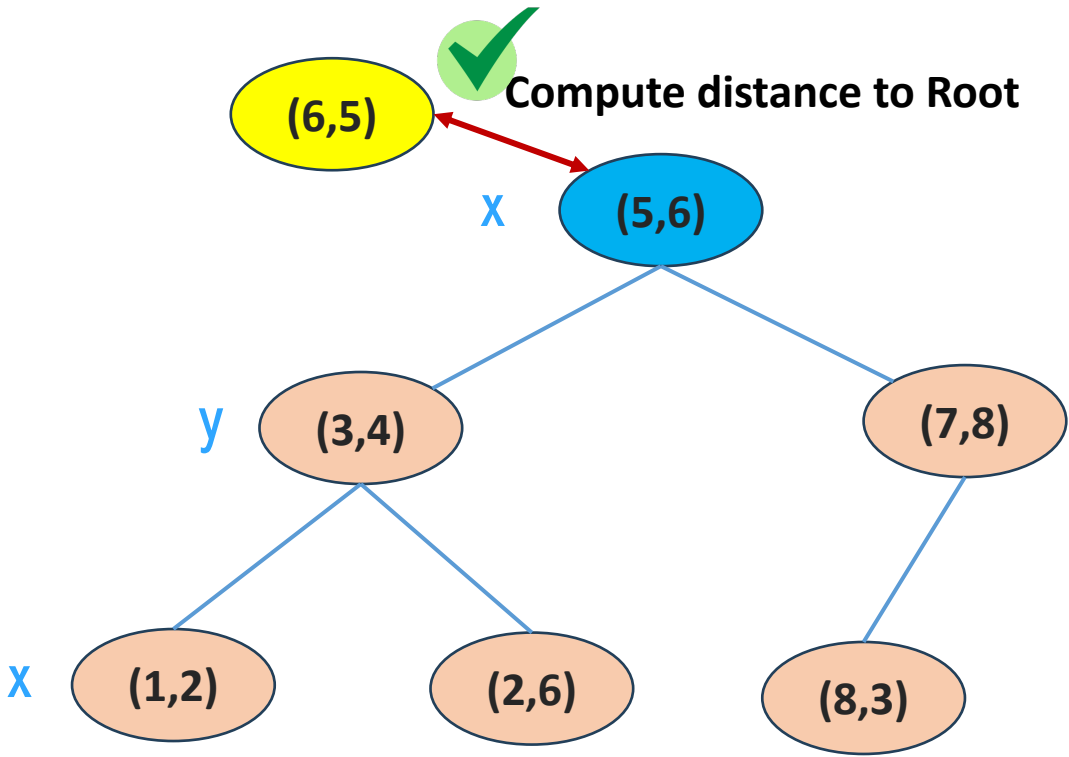
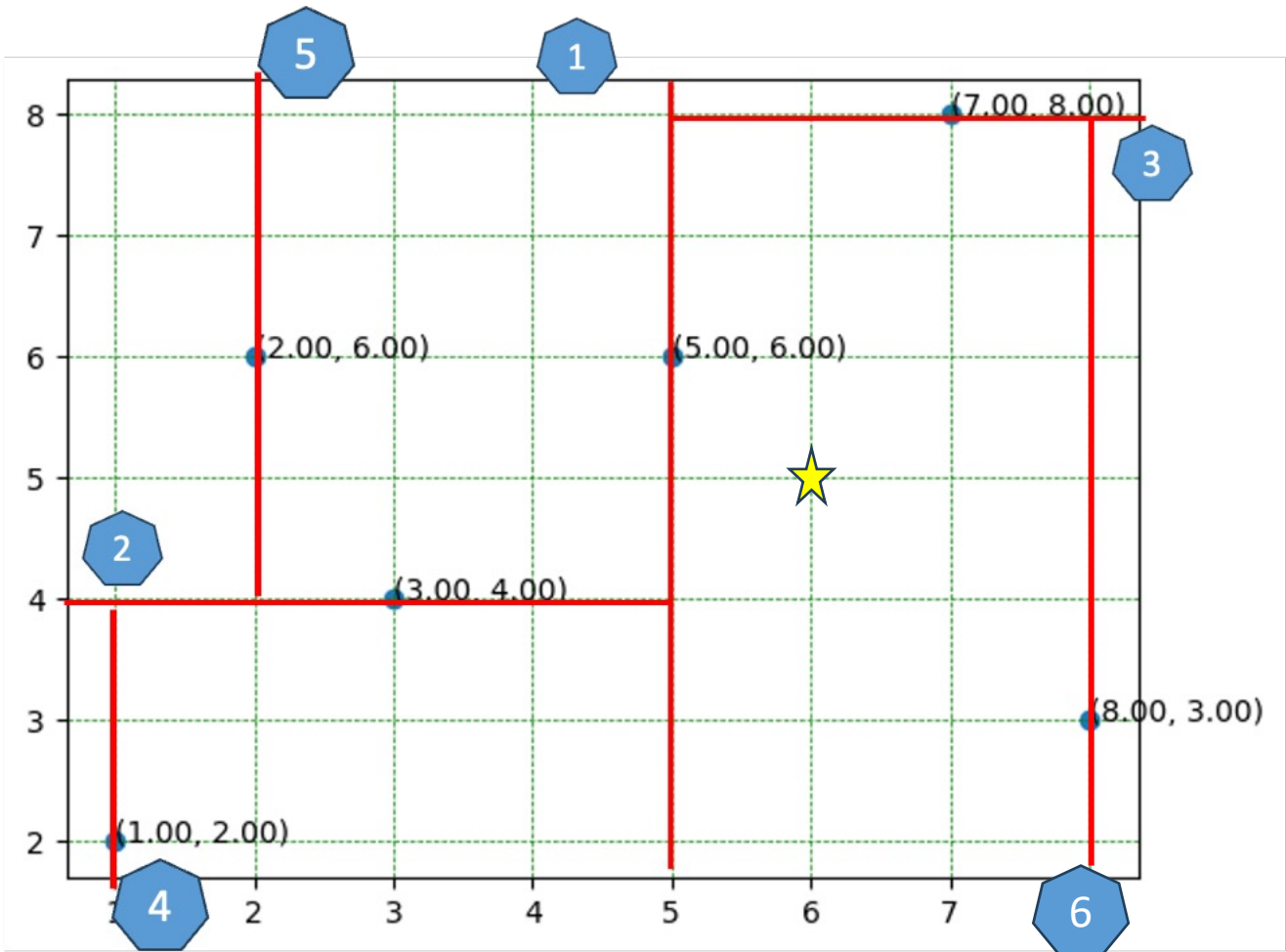


★ New datapoint

K-D Tree Implementation Solution

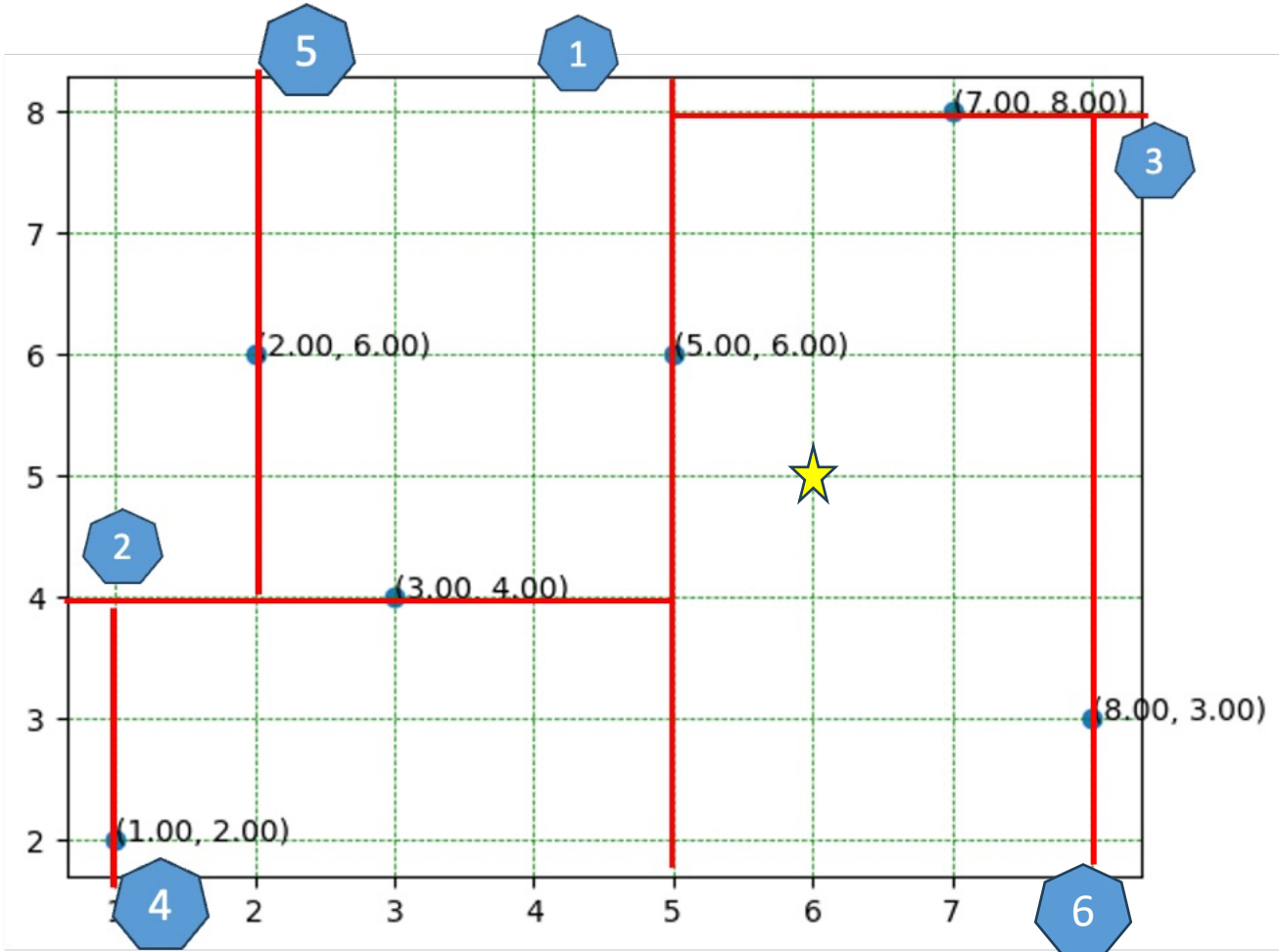


K-D Tree Implementation Solution

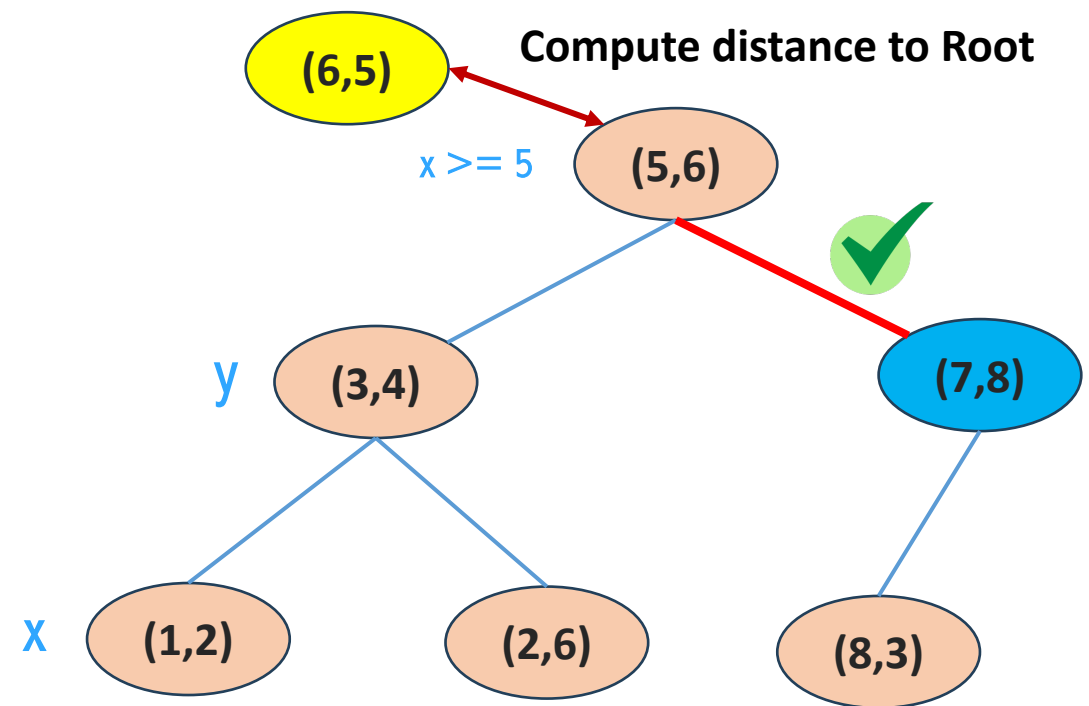


Best distance = 1.41
Best label = (5,6)

K-D Tree Implementation Solution



★ New datapoint

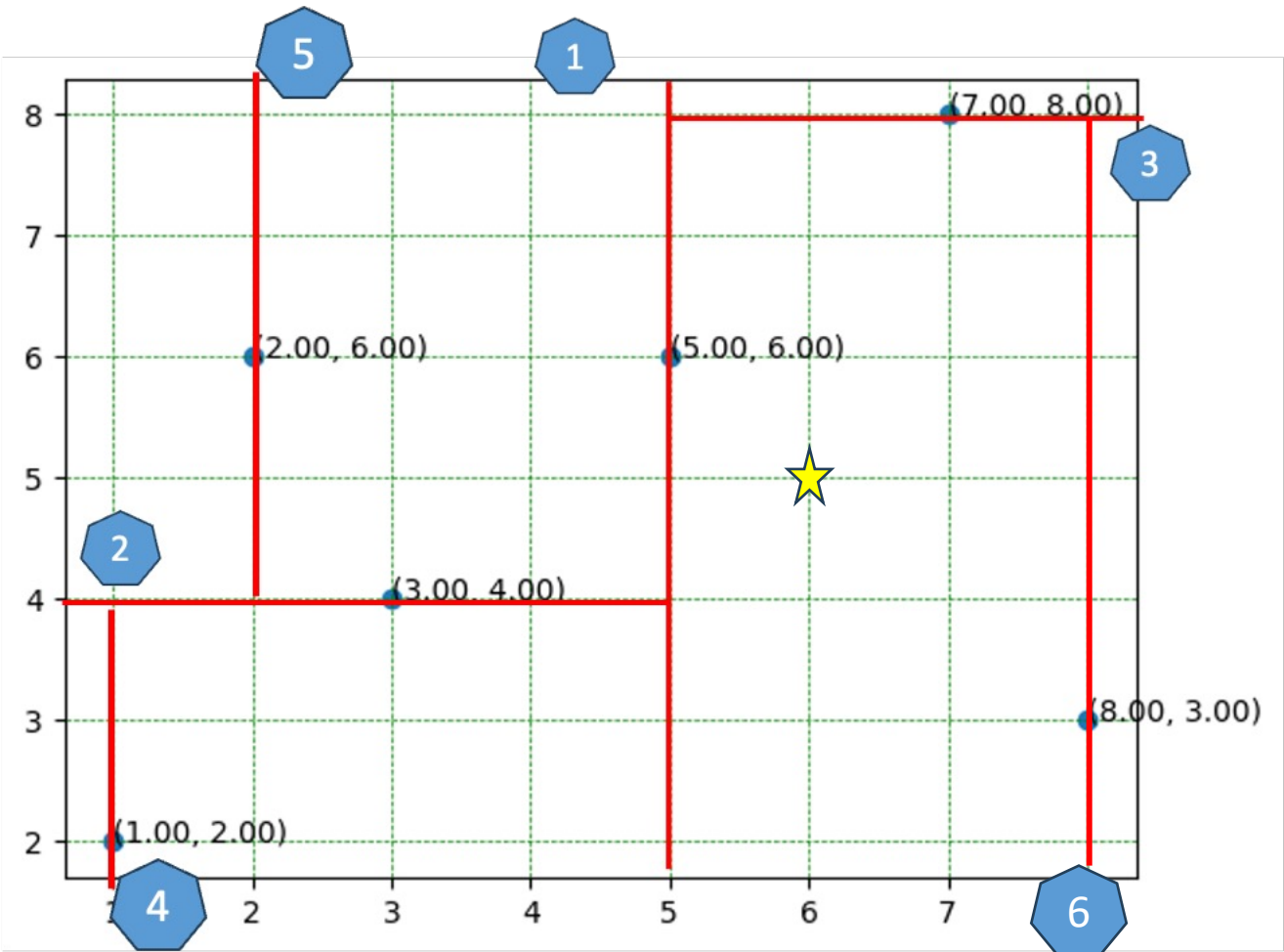


Best distance = 1.41
Best label = (5,6)

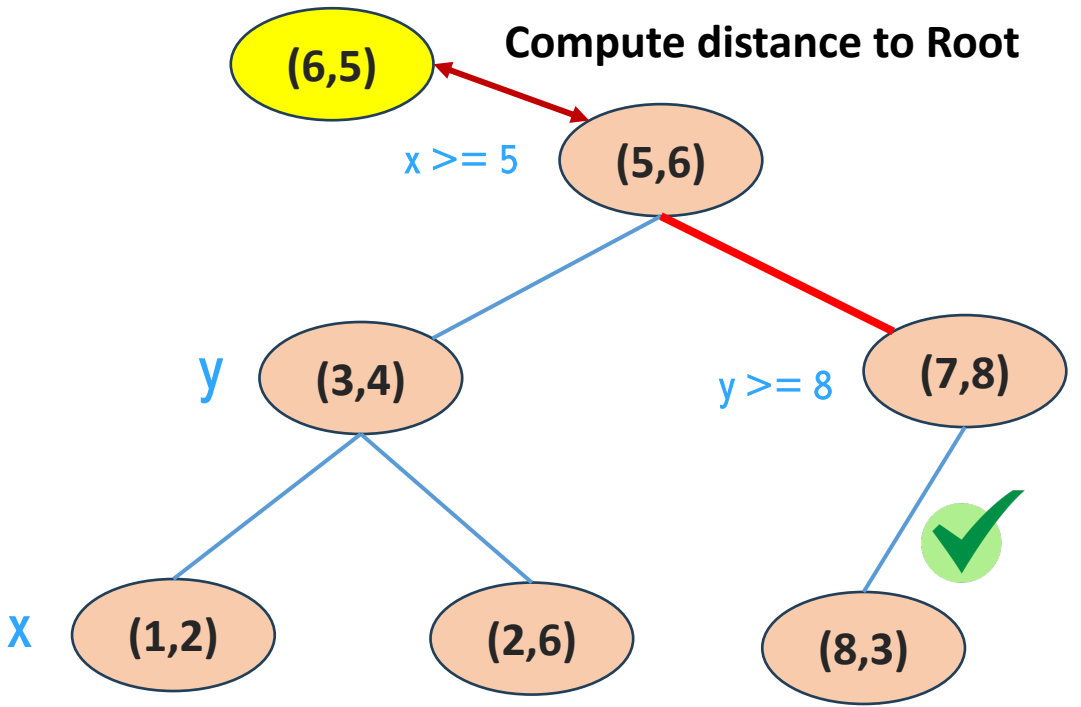
Current distance = 3.16
Current label = (5,6)

Not update the best distance

K-D Tree Implementation Solution



(6,5)
The closest points is (5,6)



Best distance = 1.41
Best label = (5,6)

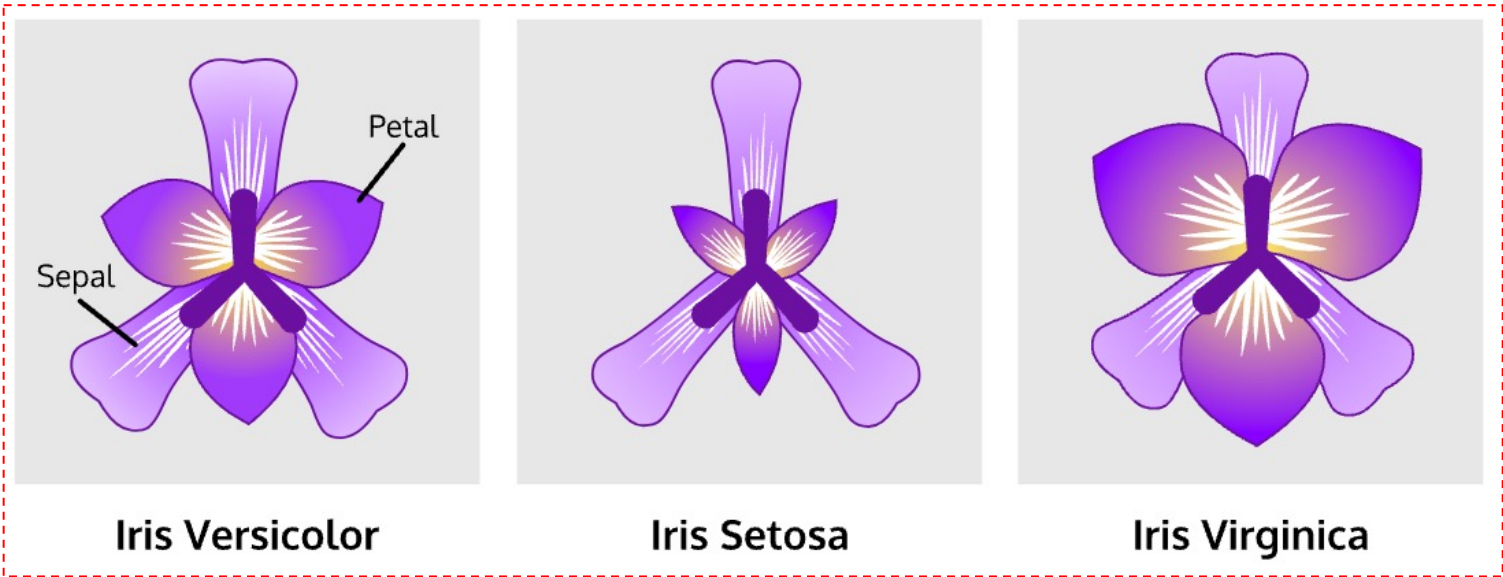
Curent distance = 2.82
Curent label = (8,3)

Not update the best distance

K-D Tree Implementation



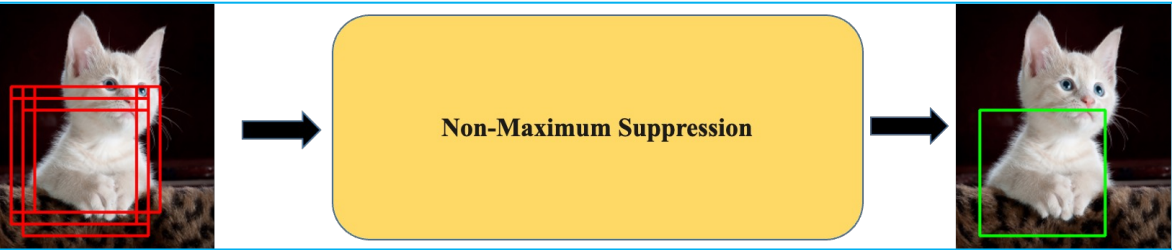
Iris Dataset Classification: Self-study



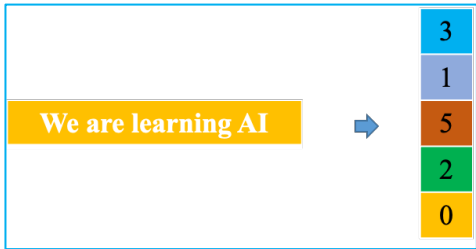
	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Summary

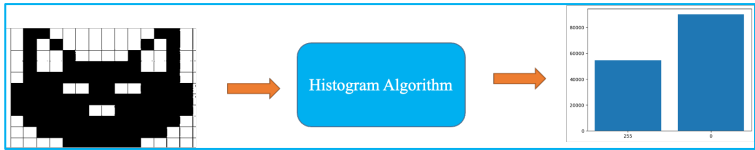
Data Structure	Ordered	Mutable	Constructor	Example
List	Yes	Yes	[] or list()	[5.7, 'yes', 5.7]
Tuple	Yes	No	() or tuple()	(5.7, 'yes', 5.7)
Set	No	Yes	{ } or set()	{5.7, 'yes' }
Dictionary	No	Yes	{ } or dict{ }	{'key': value}



IOU, NMS using Tuple



Text embedding using Set



Histogram using Dictionary

References

Problem Solving with Algorithms and Data Structures

Release 3.0

Brad Miller, David Ranum

September 22, 2013

Python Data Structures and Algorithms

Improve the performance and speed of your applications



Packt>

